

Module 1: Exploratory Data Analysis

Overview

- This module will introduce you to vectorized data structures and the Pandas library.
- You will learn how to explore tabular data sets.
- Pandas 3.x

A Word On Vibe Coding...

- We fully expect (and encourage) you to use AI to generate the answers to the labs and exercises.
- However, understanding the structure and mechanism of Pandas will allow you to give better direction to AI.
- We therefore would suggest writing code manually until you are comfortable with the concepts behind Pandas.
- Don't just cut/paste the instructions into an LLM.

Why do this at all?

- While you *can* send data to an LLM, there are issues with doing so:
  - There are fundamental limits as to how much data you can send to an LLM.
  - Analyzing data with an LLM is slow and costly.
  - No way to independently audit or verify the results.
- LLMs can and will produce inaccurate information.

A Better Approach with AI

- Include a description of the data's schema and perhaps a sample in your AI prompt.
- Have the LLM generate python code to accomplish your task
- More on this later...

Vectorized Data Structures

What are they?

Vectorized Data Structures let you perform operations on your data all at once

Advantages of Vectorized Data Structures

- More readable and simpler code
- Allows the module to optimize for CPU and Memory usage
- Pass serialized objects from one system to another. (More on this later)

```
for cell in row:
    half cell = cell / 2
```

```
for x in range(0):
    def( data ):
        data[x] = data[x] + 1
```

```
for cell in row:
    half = cell / 2
```

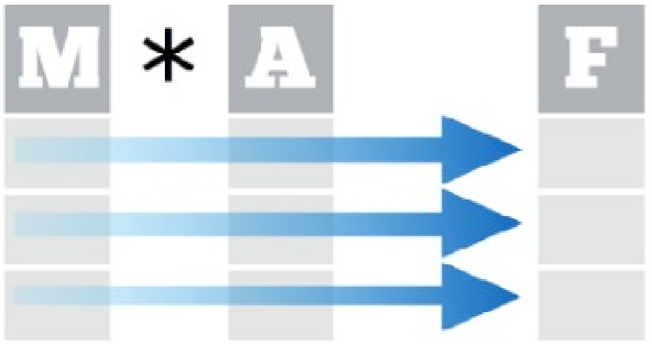
```
for x in range(0, len(data)):
    data[x] = data[x] + 1
```

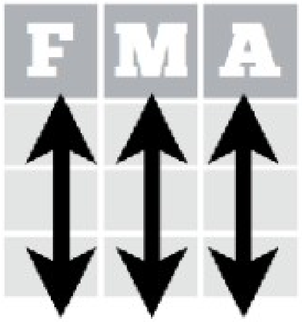
```
for x in range(0,
len( data )):
    data[x] = data[x] + 1
```

```
for cell in row:
  half_cell = cell / 2
```

```
odds = evens +
1
```

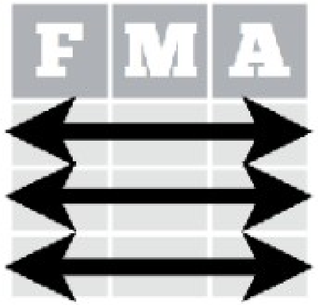
F = M * A





Each **variable** is saved in its own **column**

&



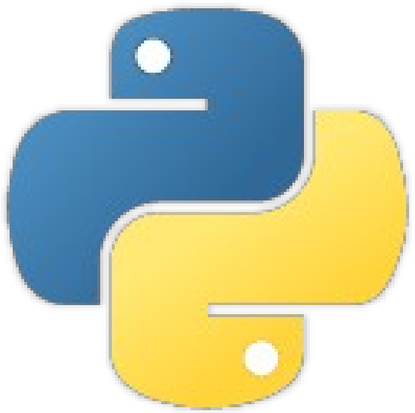
Each **observation** is saved in its own **row**

No loops

No loops



No loops



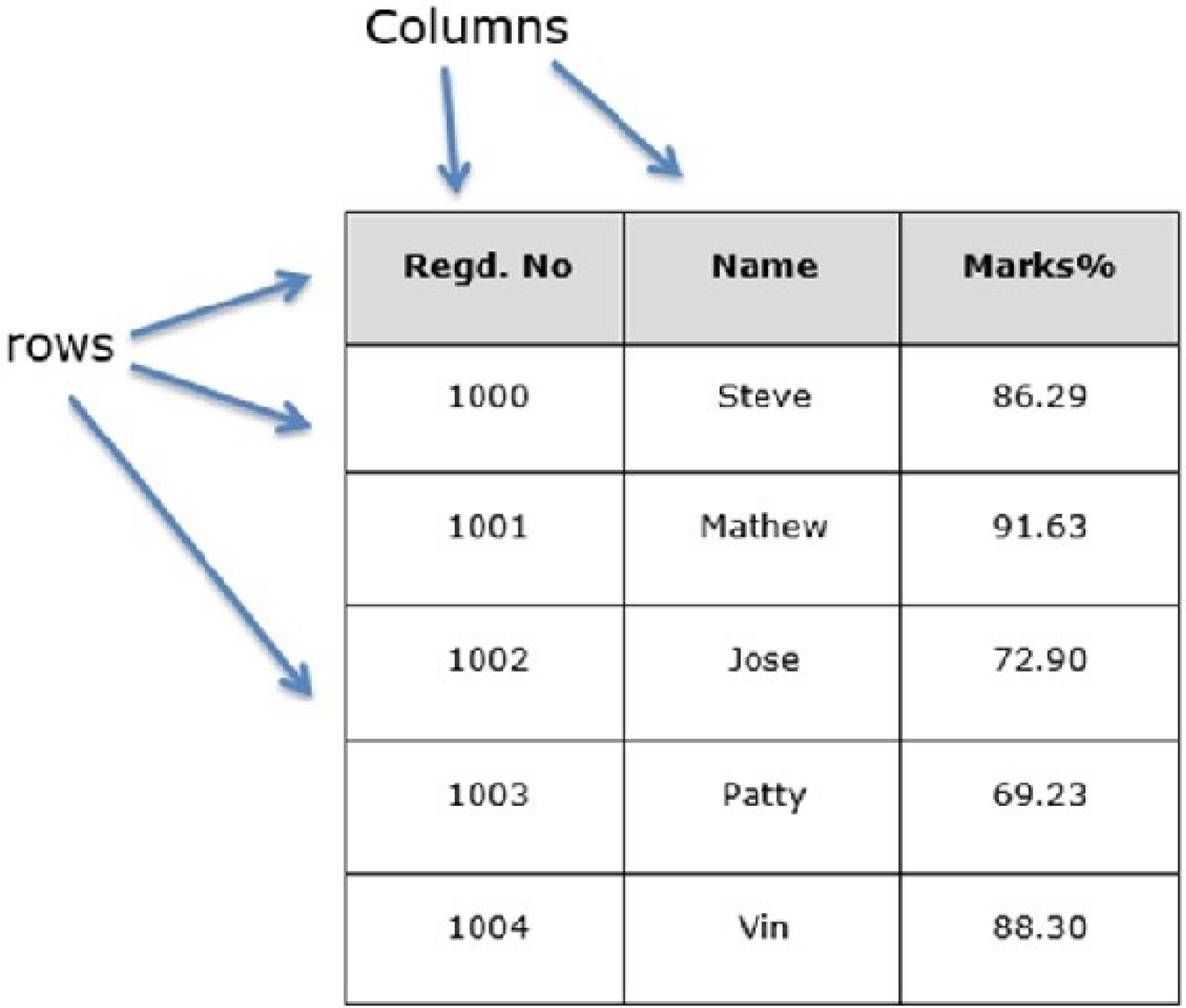




What is Pandas?

- Pandas is the standard OSS data manipulation library for Python
- At its core, it is a collection of vectorized data structures.
- Pandas is the foundation of the Python data science ecosystem, so understanding it is key to being effective at data science using Python.
- In Jan 2026, Pandas 3.0 was released which had some major new features.

Dimensions	Name	Description
1	Series	Indexed 1 dimensional data structure
1	Timeseries	Series using timestamps as an index
2	DataFrame	A two dimensional table



Regd. No	Name	Marks%
1000	Steve	86.29
1001	Mathew	91.63
1002	Jose	72.90
1003	Patty	69.23
1004	Vin	88.30

Image Source: https://www.tutorialspoint.com/python_pandas/python_pandas_dataframe.htm

Libraries in Python

What is a Python library?

- Python libraries are also called modules
- Python has large set of built-in libraries
- Can contain classes, functions or variables
- Imported into a script or session

```
import ipaddress
```

Python library: full name

Functions defined in a **library** are called using **dot notation** when importing the full library.

library name + period + function name

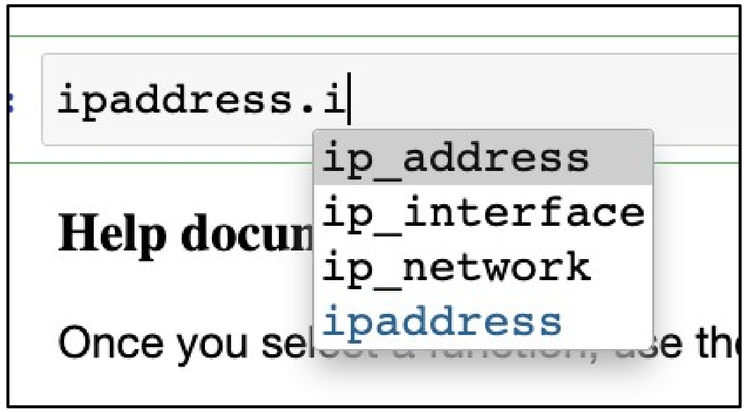
```
import ipaddress
user_address = ipaddress.ip_address("0:0:0:0:0:FFFF:185.15.58.226")
user_address.is_private
```

```
False
```

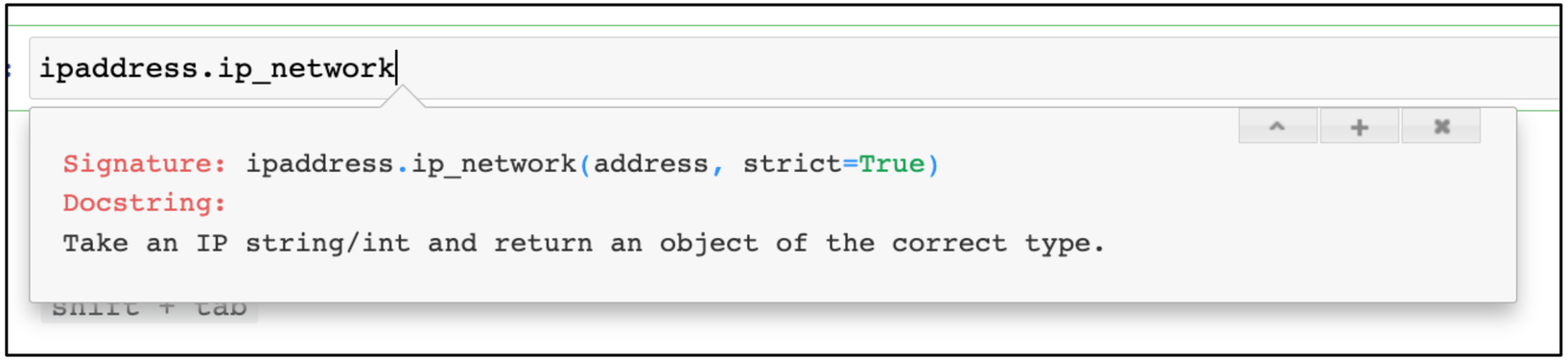
Note: try using use tab completion after typing the dot and the letter ‘i’ to get a tool tip of possible functions that start with ‘i’ from the ipaddress library.

Python library: Jupyter shortcuts

Tab completion



Shift + Tab = documentation



Python library: alias

Use an **alias** to avoid typing the full name of the **library** each time a **function** is called.

```
import ipaddress as ip
user_address = ip.ip_address("185.15.58.226")
```

```
import pandas as pd
raw_empty_data_frame = pd.DataFrame()
```

Python library: specific functions

When importing specific **functions** from a **library**, the function name can be called without the dot notation if we import them specifically using:

```
from library import function1, function2
```

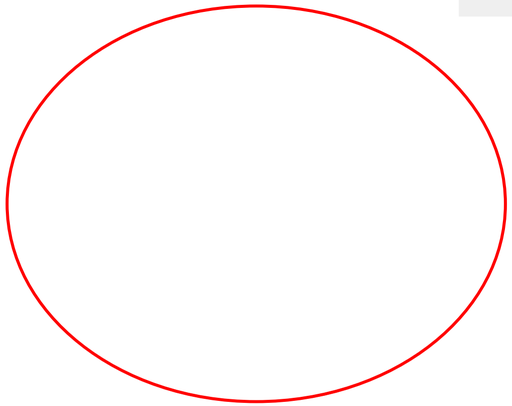
```
from ipaddress import ip_address, ip_network, ip_interface
ip_interface("0:0:0:0:0:FFFF:185.15.58.226")
```

Python library: wild card

DON’T DO THIS!!!

When importing specific **functions** from a **library**, all the functions, variables, etc can be imported using the **wildcard**.

```
from ipaddress import *
ip_network("192.0.2.0/24")
```

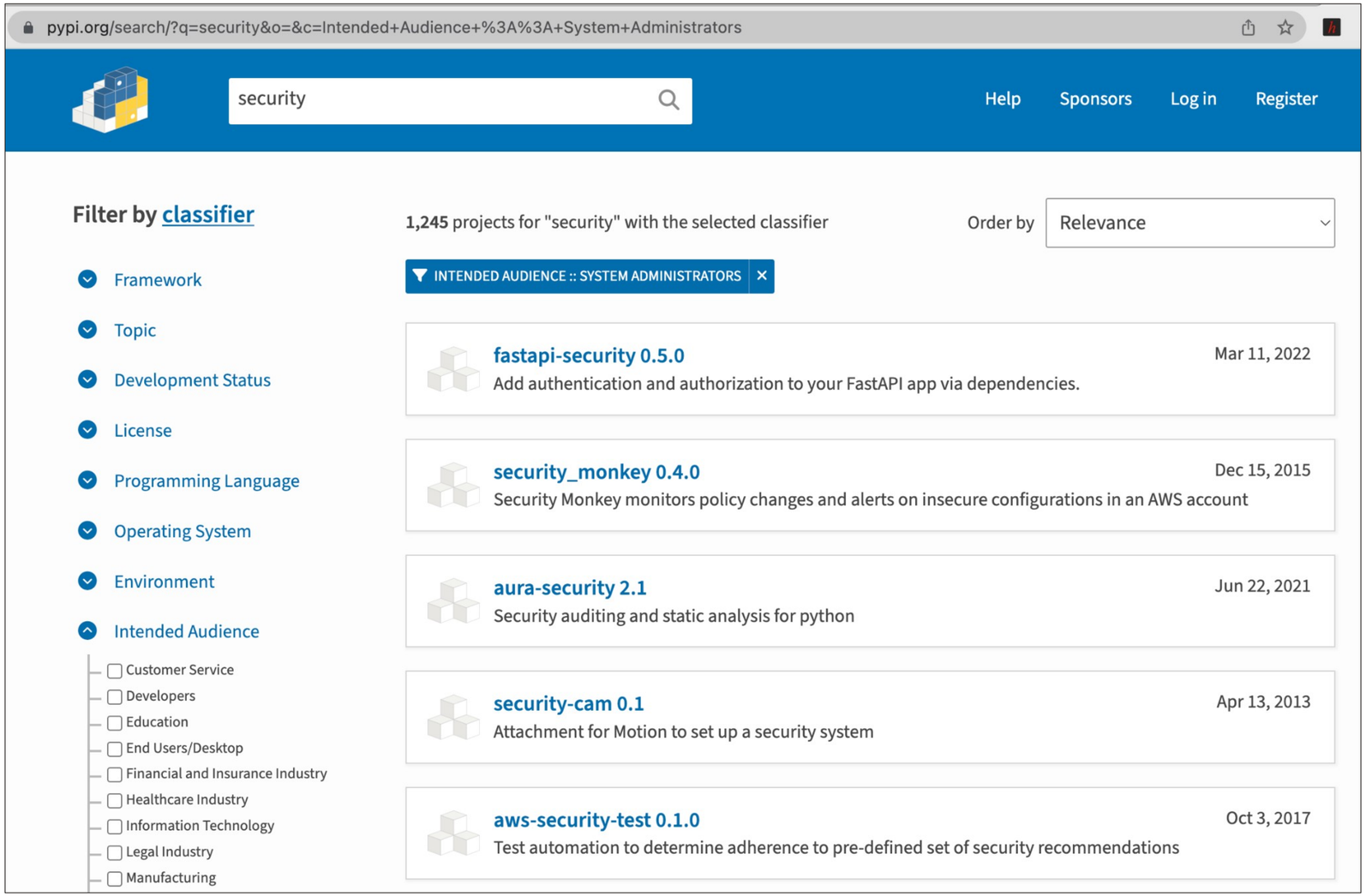


Note: this is bad practice because you don’t always know what you have imported and this can cause naming conflicts, make code harder to read, and clutter the namespace with things you won’t use. Despite this, you will see it used all the time.

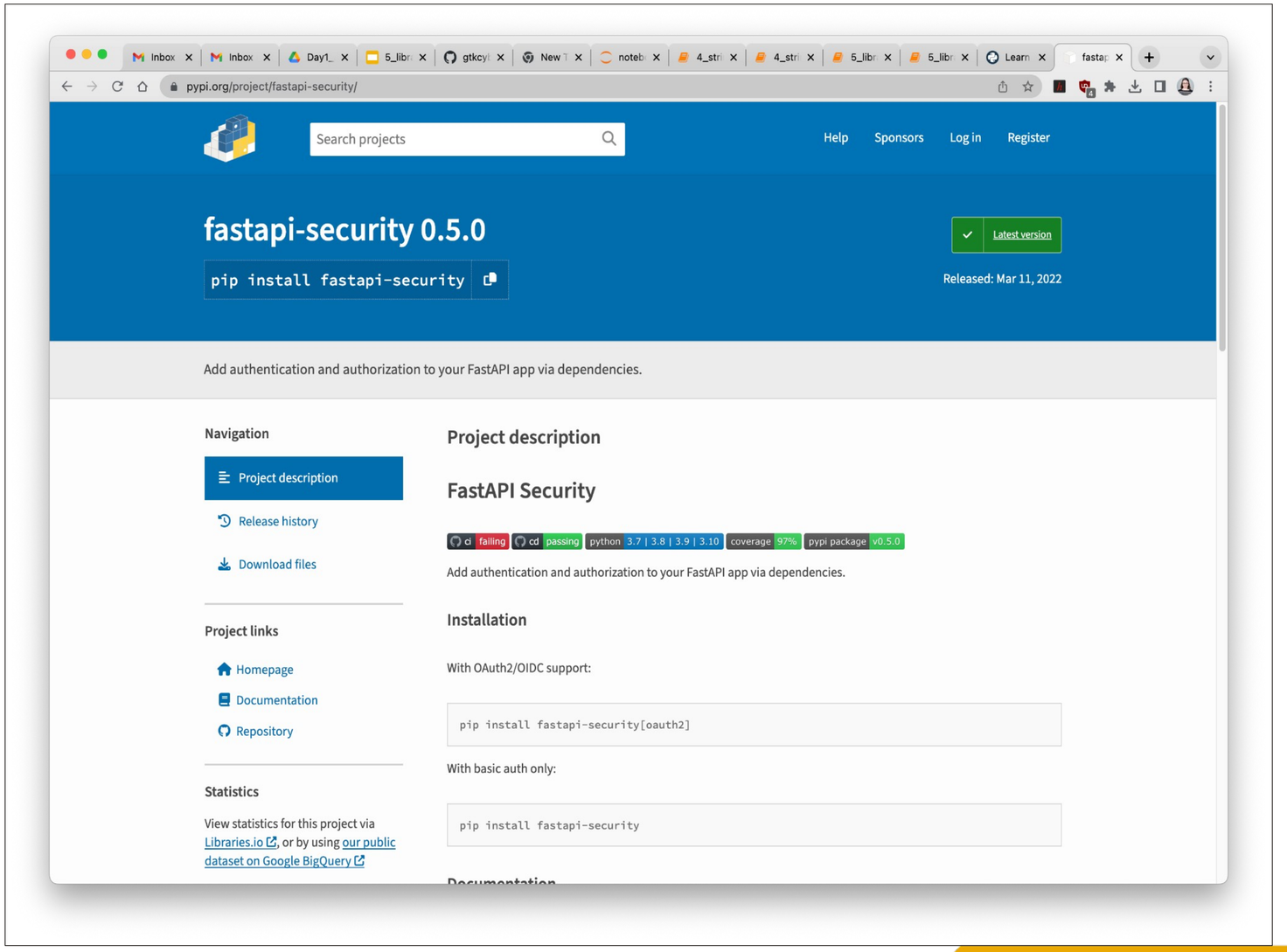
Third party Python library

- Third party library = written by someone other than Python developers
- Many useful third party libraries for data science tasks
- Not installed with Python
- Can be installed into the Python environment (pip, conda) or directly in a Jupyter Notebook (pip)
- Download from PyPI (Python Package Index) (pypi.org)
- Imported/used the same way the built-in libraries are imported

Third party Python library



Third party Python library



The Series Object

A Series is like a list

A Series is like a list or a dictionary

A Series is like a list or a dictionary but BETTER!!

The Series is the primary building block for all the other data structures we will discuss

Python libraries: Common for Data Sci

```
 import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Creating a Series

```
import pandas as pd
myData  =
pd.Series( <data> )
```

Creating a Series

Creating a Series

```
import pandas as pd
series1 = pd.Series( [‘a','b','c','d','e']
)
```

Creating a Series

```
import pandas as pd
series1 = pd.Series( [‘a','b','c','d','e']
)
```

```
series2 = pd.Series( {‘firstName':'Charles',
                      ‘lastName’ : ‘Givre’} )
```

Pandas Series

Pandas Series

```
series1 =
pd.Series( ['a','b','c','d','e'] )
print(series1)
```

Pandas Series

```
series1 =
pd.Series( ['a','b','c','d','e'] )
print(series1)
```

```
0    a
1    b
2    c
3    d
4    e
dtype: object
```

Pandas Series: single elements

d

Pandas Series: single elements

```
series1 =
pd.Series( ['a','b','c','d','e'] )
print(series1[3])
```

```
d
```

Pandas Series: slicing

```
1    b
2    c
dtype: object
```

Pandas Series: slicing

```
series1 =
pd.Series( ['a','b','c','d','e'] )
print(series1[1:3])
```

```
1    b
2    c
dtype: object
```

Accessing a Series

```
series1 =
pd.Series( ['a','b','c'
,'d','e'])
```

```
print( series1[1:3] )
```

```
1    b
2    c
dtype: object
```

Accessing a Series

```
series1 =
pd.Series( ['a','b','c'
    ,'d','e'])
```

```
print( series1[1:3] )
```

```
1    b
2    c
dtype: object
```

```
series2 = series1[1:3]
series2.reset_index( drop=True )
```

Accessing a Series

```
Charles
```

Accessing a Series

```
series2 = pd.Series( {'firstName':'Charles',
                      'lastName' : 'Givre'} )
print(series2['firstName'])
```

```
Charles
```

Accessing a Series

```
firstName    Charles
lastName       Givre
dtype: object
```

Accessing a Series

```
series2 = pd.Series( {‘firstName':'Charles',
                      ‘lastName’ : ‘Givre’} )
print(series2[[‘firstName’,’lastName’]])
```

```
firstName    Charles
lastName       Givre
dtype: object
```

Accessing a Series

```
Charles
```

Accessing a Series

```
series2 = pd.Series( {‘firstName':'Charles',
                      ‘lastName’ : ‘Givre’} )
print(series2[0])
```

```
Charles
```

Accessing a Series

```
randomNumbers = pd.Series(

        np.random.random_integers(1, 100, 50))
randomNumbers.head()
```

```
0    48
1    34
2    84
3    85
4    58
dtype: int64
```

Accessing a Series

```
randomNumbers = pd.Series(

   np.random.random_integers(1, 100, 50) )
randomNumbers.tail( 7 )
```

```
43    66
44    66
45    43
46    55
47    99
48    82
49    19
dtype: int64
```

Filtering Data in a Series

```
randomNumbers [ <boolean condition> ]
randomNumbers [ randomNumbers < 10 ]
```

```
12    5
21    2
24    1
27    1
29    1
dtype:
int64
```

Filtering Data in a Series

```
record.str.contains('Cha')
firstname     True
lastname     False
middle       False
dtype: bool
record[ record.str.contains('Cha')
]
firstname    Charles
dtype: object
```

String Functions

Function	Explanation
Series.str.contains(<pattern>)	Returns true/false if text matches a pattern
Series.str.count(<pattern>)	Returns number of occurrences of a pattern in a string
Series.str.extract(<pattern>)	Returns matching groups from a string
Series.str.find(<string>)	Returns index of first occurrences of a substring (Note: not regex)
Series.str.findall(<pattern>)	Returns all occurrences of a regex
Series.str.len()	Returns the length of text
Series.str.replace(<pat>, <replace>)	Replaces matches with a replacement string

https://pandas.pydata.org/docs/user_guide/text.html#method-

Date/Time Operations

```
pd.to_datetime(<times>, format='<format string>')
```

Selected Date/Time Operations

Function	Explanation
`series.dt.year`	Returns the year of the date time
`series.dt.month`	Returns the month of the date time
`series.dt.day`	Returns the days of the date time
`series.dt.weekday`	Returns the day of the week
`series.dt.is_month_start`	Returns true if it is the first day of the month
`series.dt.strftime()`	Returns an array of formatted strings specified by a date_format

https://pandas.pydata.org/docs/user_guide/

```
combinedSeries =
series1.add( series2 )
```

```
def addTwo(n):
    return n+2
odds.apply(addTwo)
```

```
odds.apply(lambda x: x + 1 )
```

שאלות

Questions?

Exploring Your Data



Statistics: objectives

This section will cover basic statistical methods commonly used in cleaning, analyzing and visualization of data. The relationship between statistics and data science will be discussed. The python skills learned thus far will be used to practice translating a mathematical equation into code.

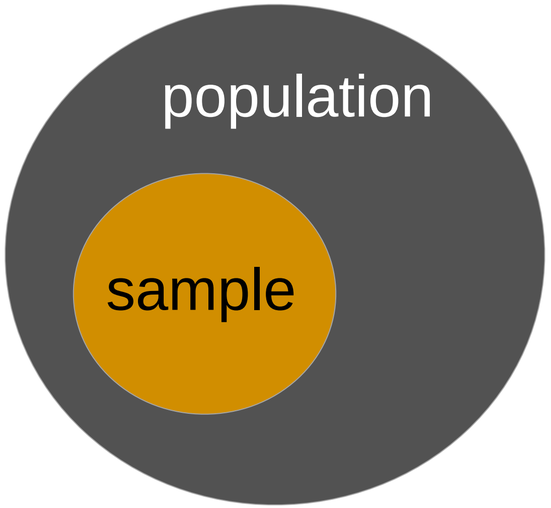
1. Explain the difference between and calculate mean, median, mode, range, standard deviation, variance of a dataset
2. Differentiate when and why it is appropriate to use mean vs median
3. Use a histogram and box plot to understand the distribution of a dataset
4. Identify outliers and decide what actions to take
5. Normalize and scale a feature and describe when and why it is necessary

Statistics: define

Probability is a mathematical tool used to study randomness. It deals with the chance (the likelihood) of an event occurring

Statistics: define

Population - a collection of persons, things, or objects under study.
To study the population, we select a **sample** - a portion (or subset) of the larger population. Study this portion (sample) to gain information about the population.
Data are the result of sampling from a population.



Statistics: variables

Variables - a characteristic or measurement for each member of a population

Numerical - numbers (weight, bytes, time)

Categorical - a category that members can be put into (user/bot, location)

Statistics: types of data

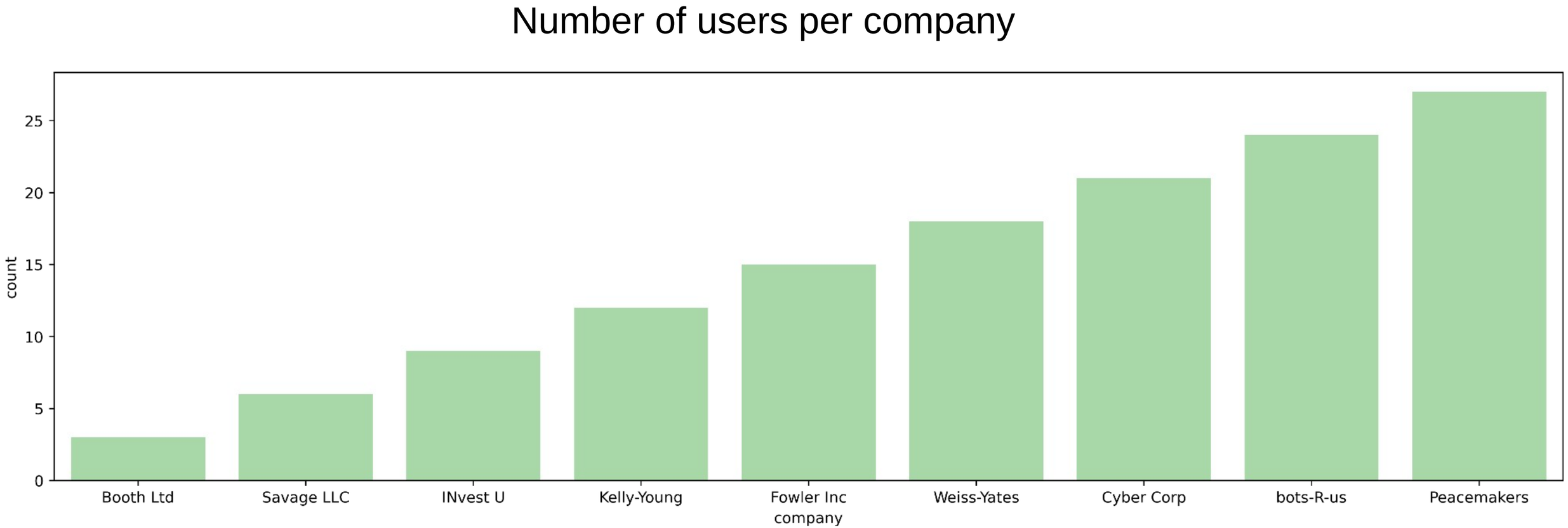
QuaLitative - aka **categorical** data, non-numeric data (user status, url, gender)
QuanTitative - numeric data

- **Discrete** - integers, like the result of counting (# of site visits, log ins)
- **Continuous** - decimal numbers, fractions (weight, amount of time)

Statistics: data example

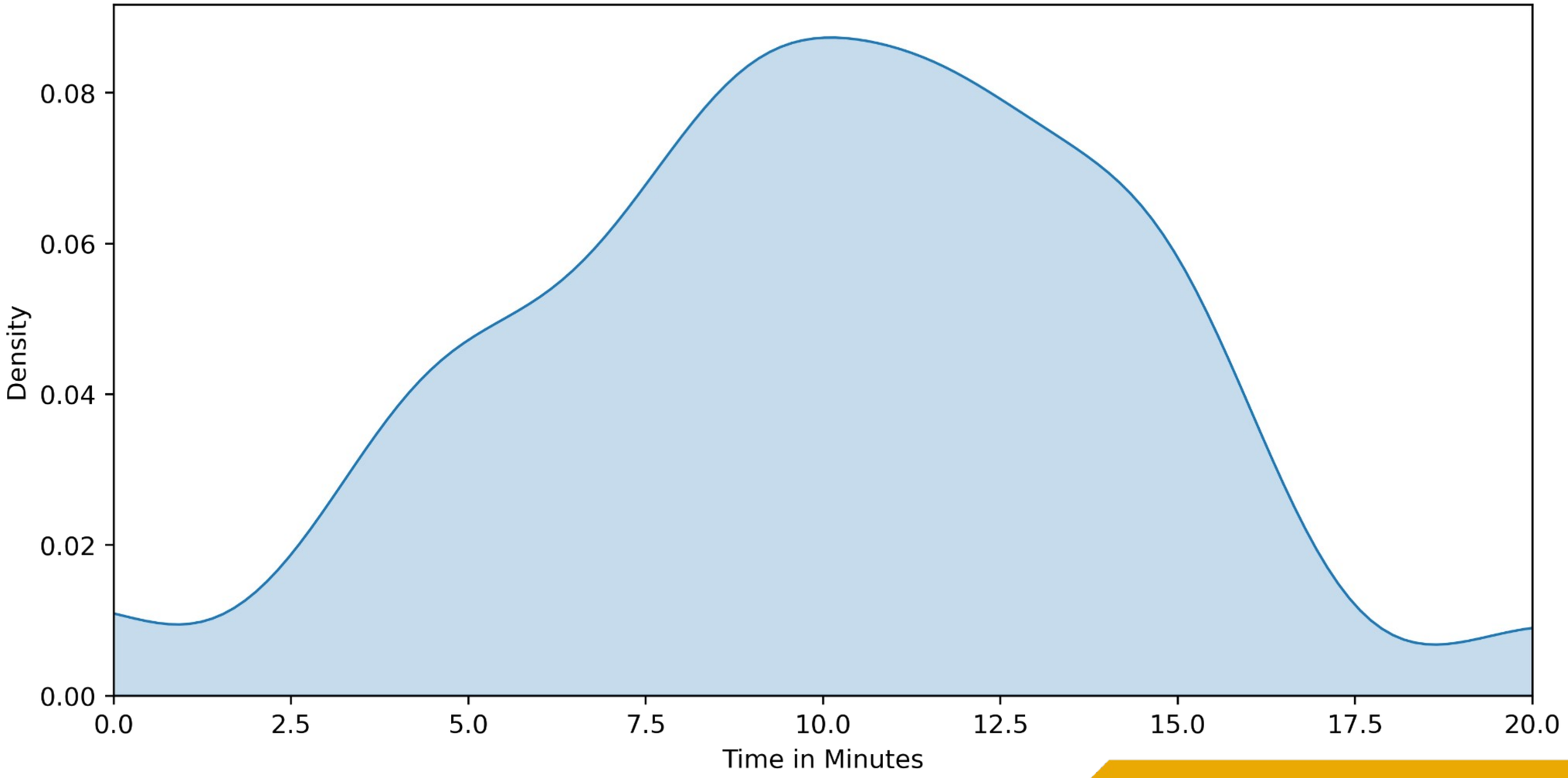
company	username	name	mail	connected
Peacemakers	susan63	Allison Hanson	denisealexander@yahoo.com	22.498529
bots-R-us	gerald76	David Martinez	hernandezmichele@yahoo.com	13.470739
Kelly-Young	christophergomez	Heather Bolton	robert00@gmail.com	13.967764
bots-R-us	mariacraig	Melissa Fisher	katierussell@hotmail.com	7.517620
Cyber Corp	lesliebarnett	Jamie Adams	morganfields@gmail.com	2.734185

Statistics: quantitative discrete



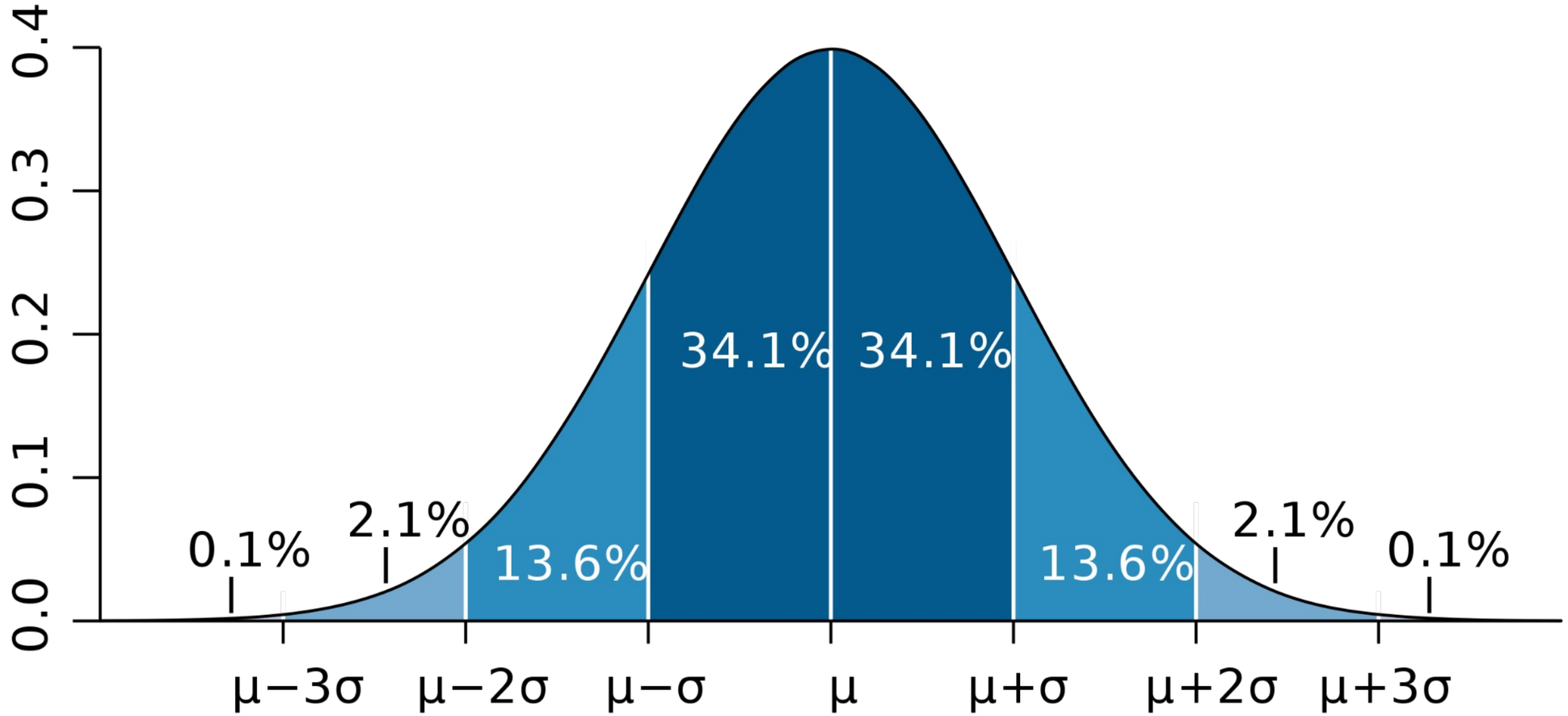
Statistics: quantitative continuous

Amount of time users were logged into server



Statistics: normal distribution

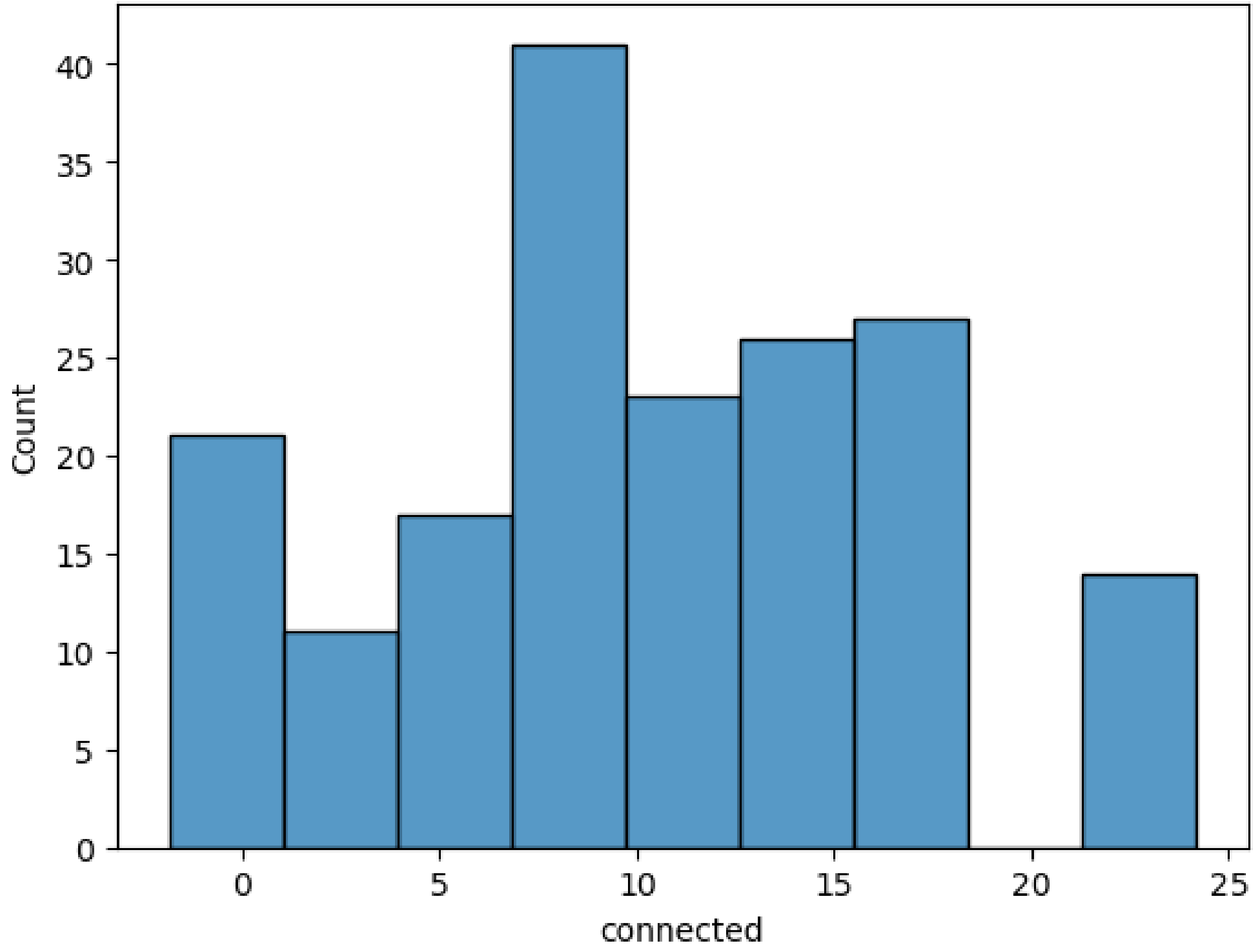
Normal distribution (Gaussian)



Ainali, CC BY-SA 3.0 <https://creativecommons.org/licenses/by-sa/3.0>, via Wikimedia Commons

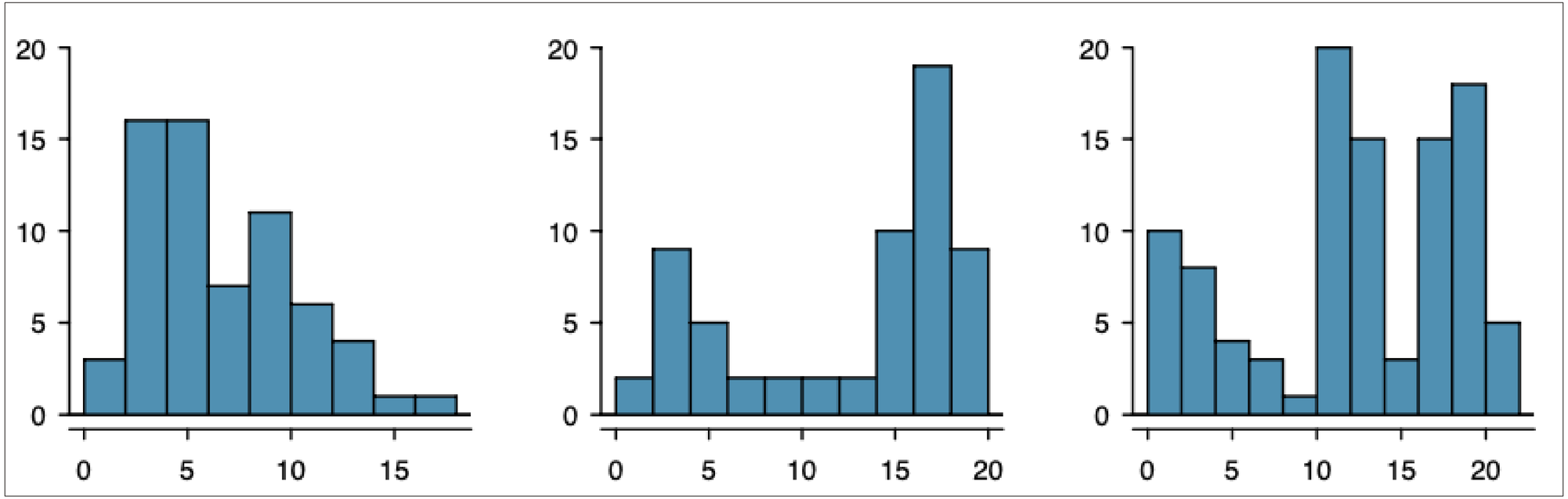
Statistics: histogram

Amount of time users were logged into server



Statistics: mode

Mode - value with the most occurrences



Unimodal Bimodal Multimodal

Statistics: mean

Mean - Measure of central tendency.

μ = population mean

$\bar{x}$ = sample mean

The mean is commonly denoted by $\bar{x}$ (x bar) the greek letter μ (mu).

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i = \frac{x_1 + x_2 + \ldots + x_n}{n}$$

We say that x bar is the sum of the values of x from 1 to n, multiplied by 1 over the number of values n.

n is the total number of data points

$x_i, i = [1, 2, 3...]$ are the values

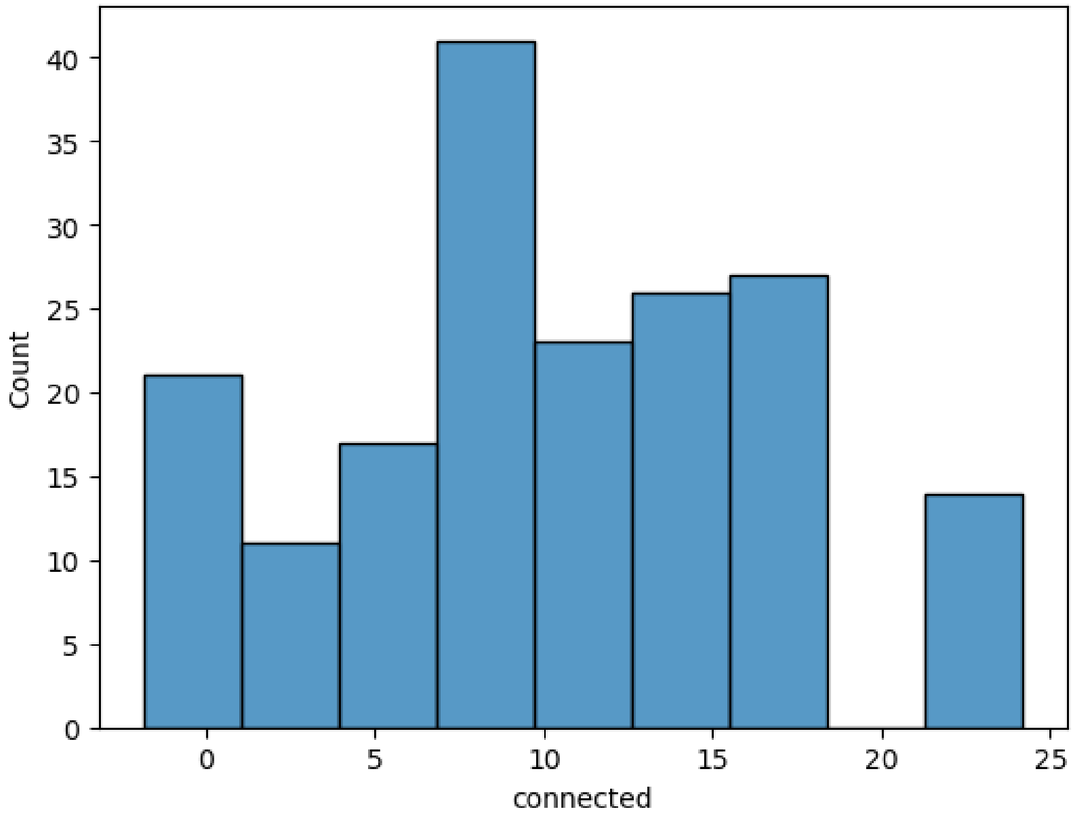
```
(3 + 2 + 9 + 4 + 1) / 5
```

Note: Sensitive to outliers

Statistics: mean

Amount of time users were logged into server

Mean = 10.11



Statistics: variation

Variation - all data has variation (spread of data around the mean)

Variance - the mean squared deviation

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

1 is s for the sample and usually

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

Standard Deviation - square root of the variance

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

1 is s for the sample and usually

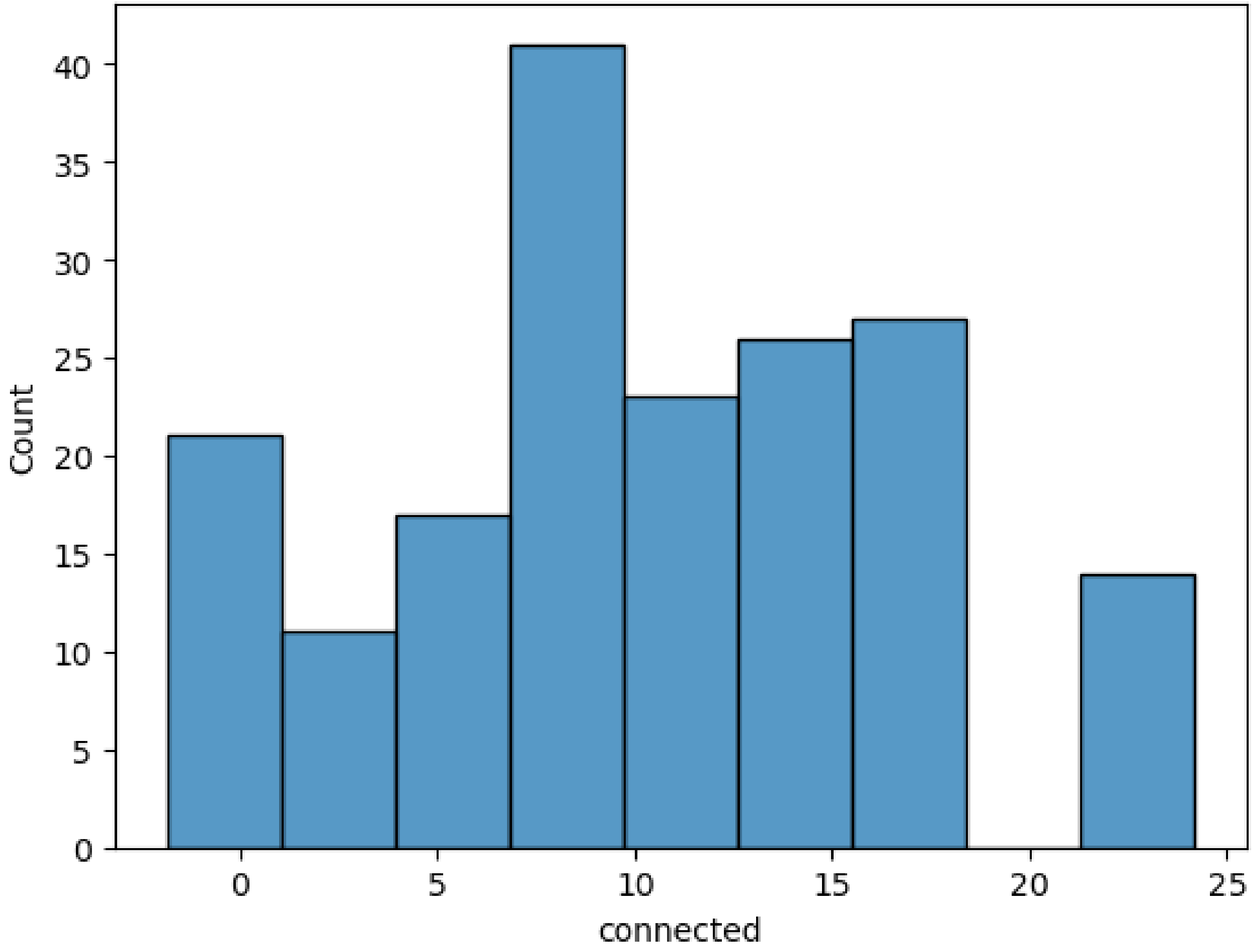
$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

Note: Sensitive to extreme values

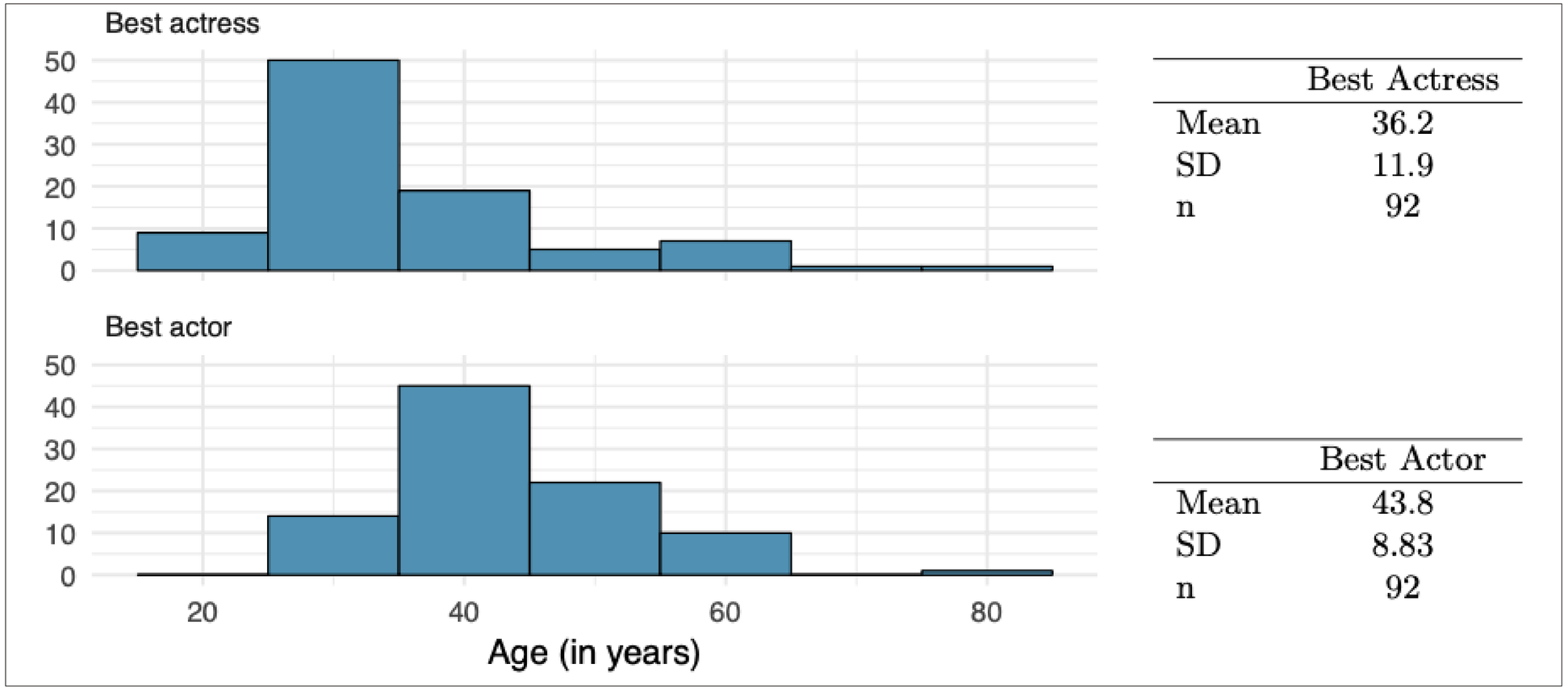
Statistics: variation

Variance = 44.73

Standard Deviation = 6.68



Statistics: mean & variation

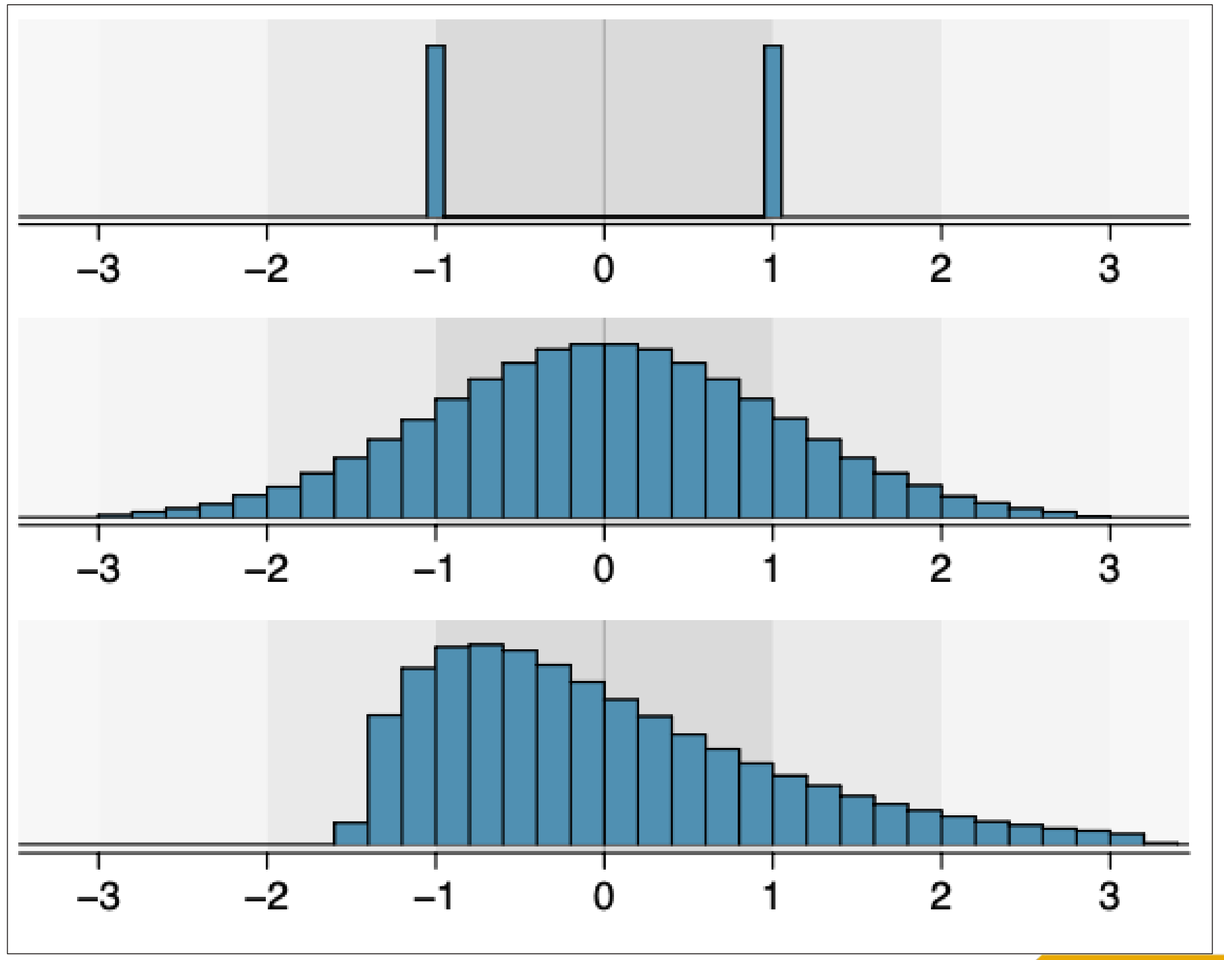


	Best Actress
Mean	36.2
SD	11.9
n	92

	Best Actor
Mean	43.8
SD	8.83
n	92

Statistics: median

Median - Middle value separating the greater and lesser halves of a data set



Statistics: median

Median - Middle value separating the greater and lesser halves of a data set

```
s_list = [8,3,1,8,3,9,2,0]
```

Sort the values

```
s_list = [0,1,2,3,3,8,8,9]
```

middle values

Note: If number of values is even, take mean of middle 2 numbers

```
median = (3+3)/2 = 3
```

Intro Stats: Covariance

Covariance, like the variance, is a measure of spread, however it also measures how closely two datasets track each other.

- **Covariance** is a squared quantity, so it is not on the same scale as the mean
- **Covariance** of different pairs of variables can have completely different scales

$$Cov(x, y) = \frac{\sum_{i=1}^{n}(x_i - \mu_x)(y_i - \mu_y)}{n}$$

Tukey 5 Number Summary

- **Minimum:** The smallest value in the dataset
- **Lower Quartile:** Smallest 25% of the dataset
- **The Median:** The middle value of the dataset
- **Upper Quartile:** The largest 25% of the dataset
- **Maximum:** The largest value in the dataset



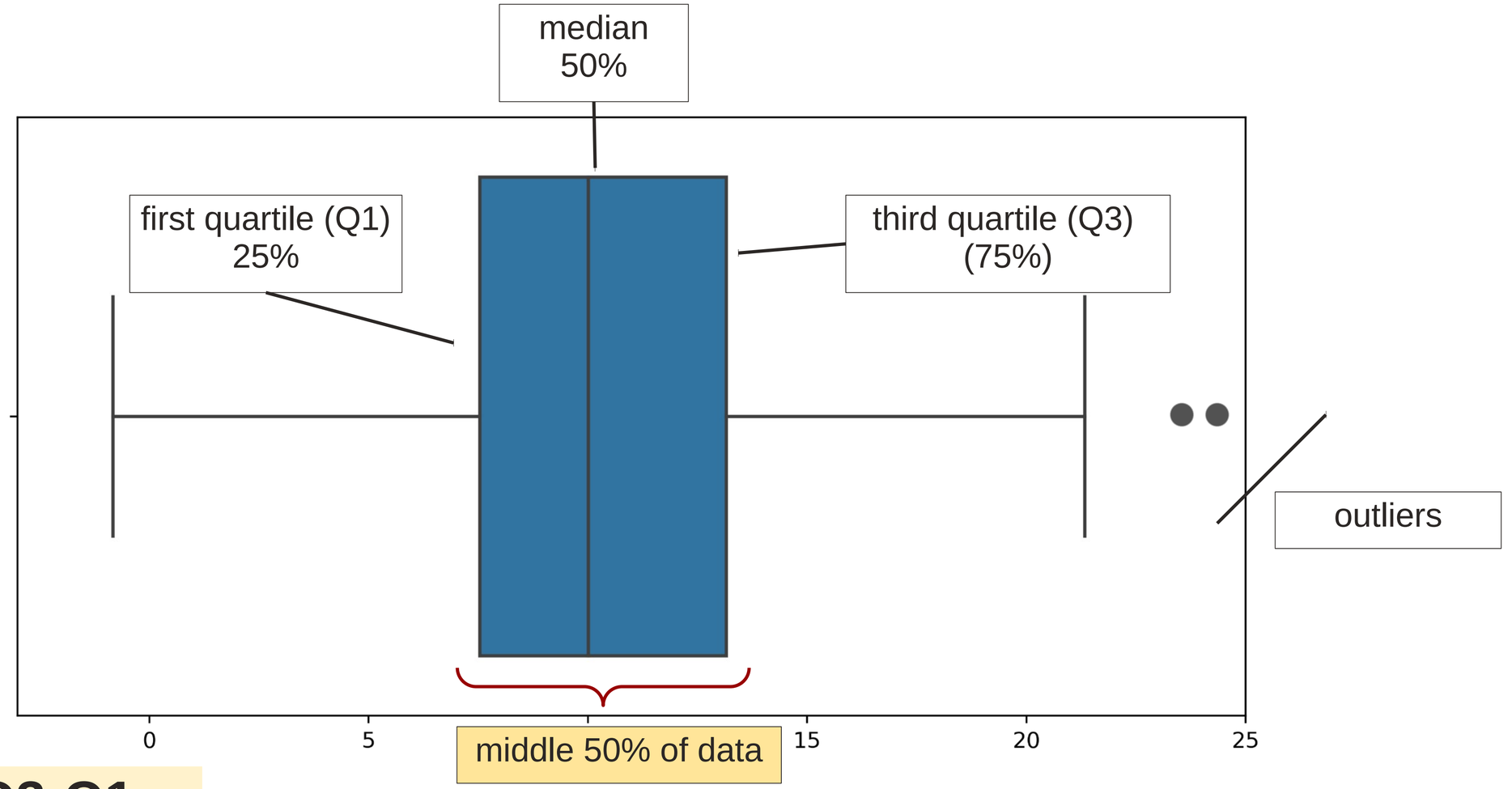
Intro Stats: IQR

The **interquartile range** is a measure of where the "middle fifty" is in a data set. Where a range is a measure of where the beginning and end are in a set, **an interquartile range is a measure of where the bulk of the values lie.**

```
IQR = data.quantile(0.75) -
data.quantile(0.25)
```

http://www.statisticshowto.com/probability-and-statistics/interquartile-range/

Statistics: IQR (interquartile range)

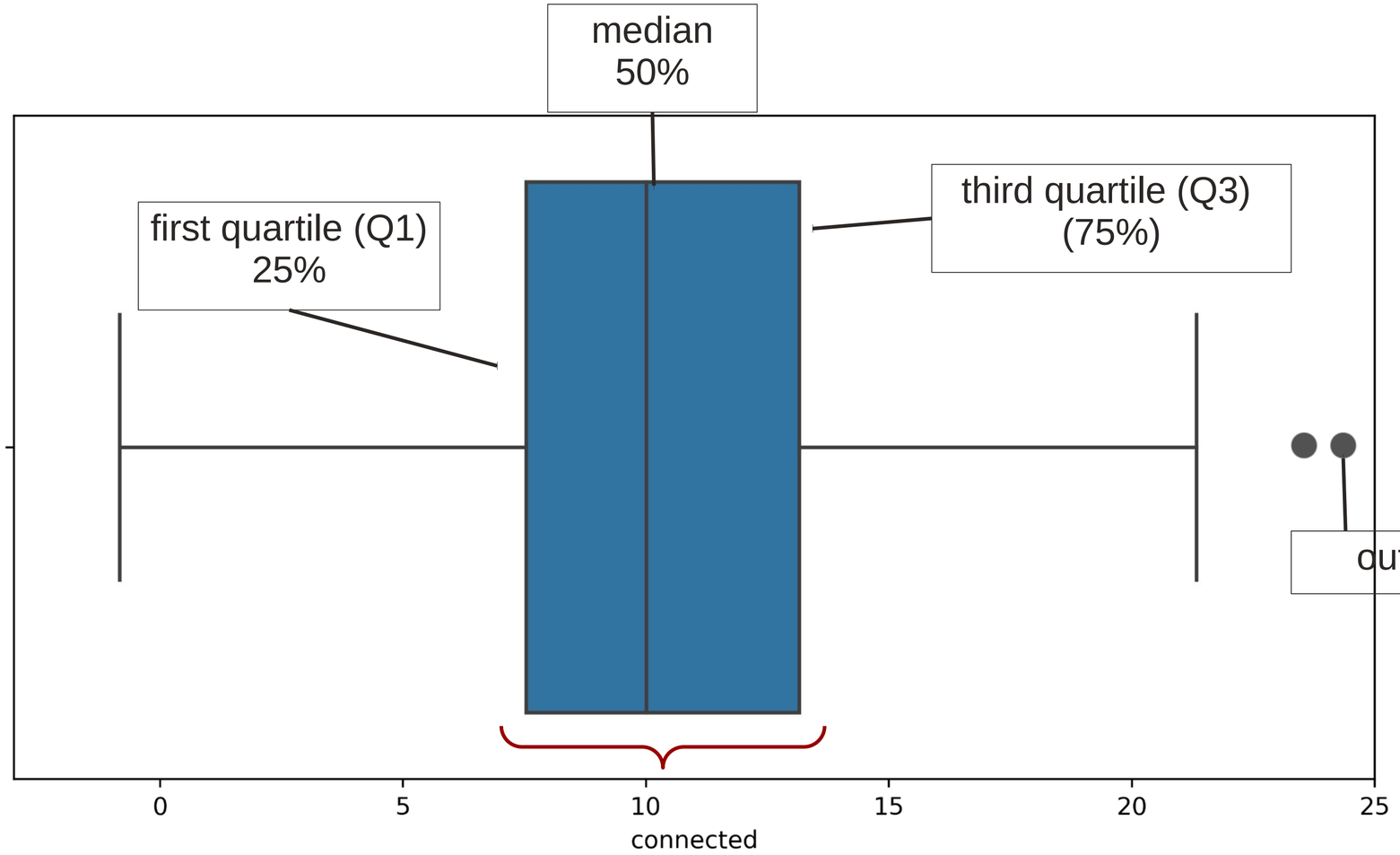


IQR = Q3-Q1

Statistics: IQR (interquartile range)

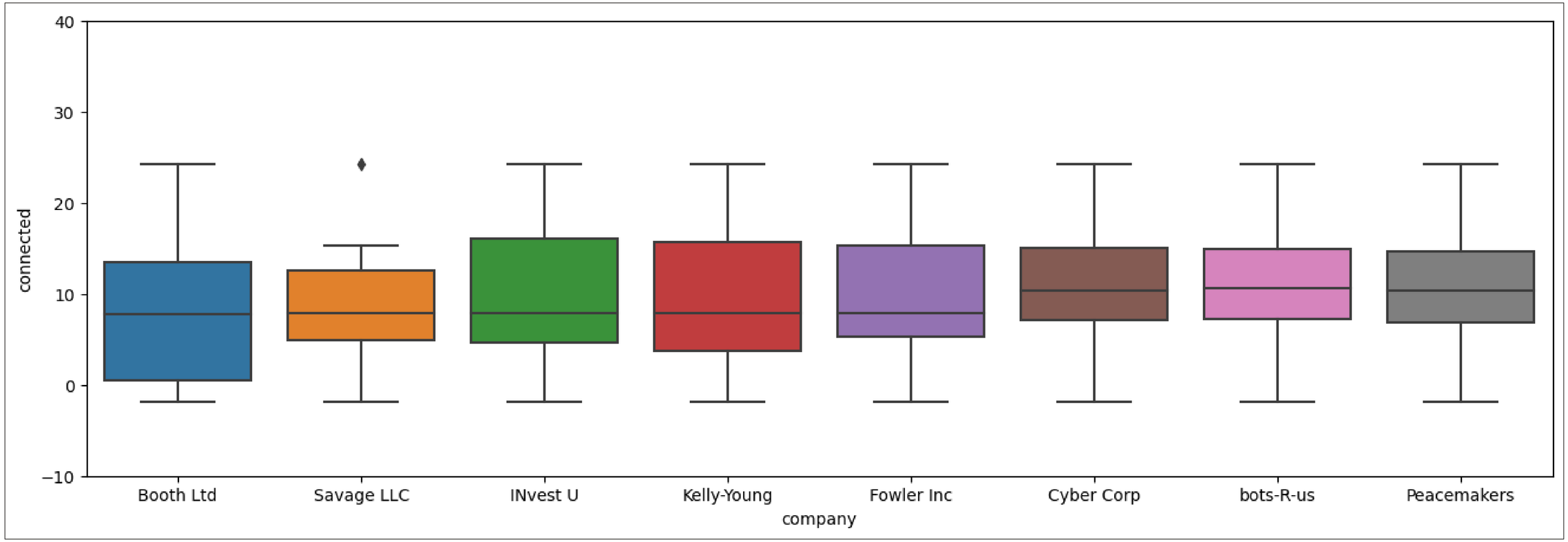
```
users.connected.describe()
```

```
count    180.000000
mean      10.111818
std        6.688669
min       -1.869387
25%        5.307170
50%        9.326331
75%       15.352434
max       24.194889
```

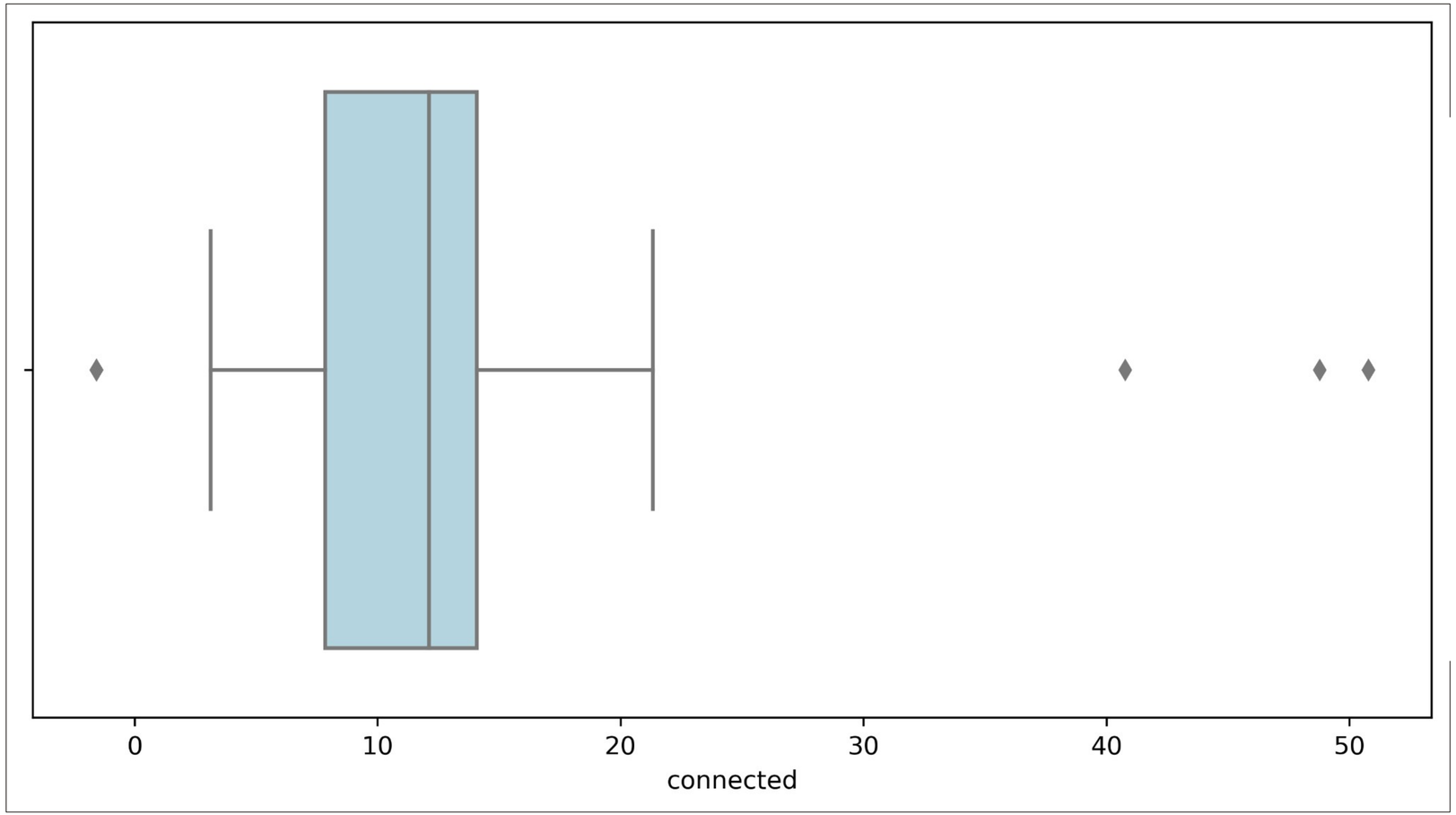


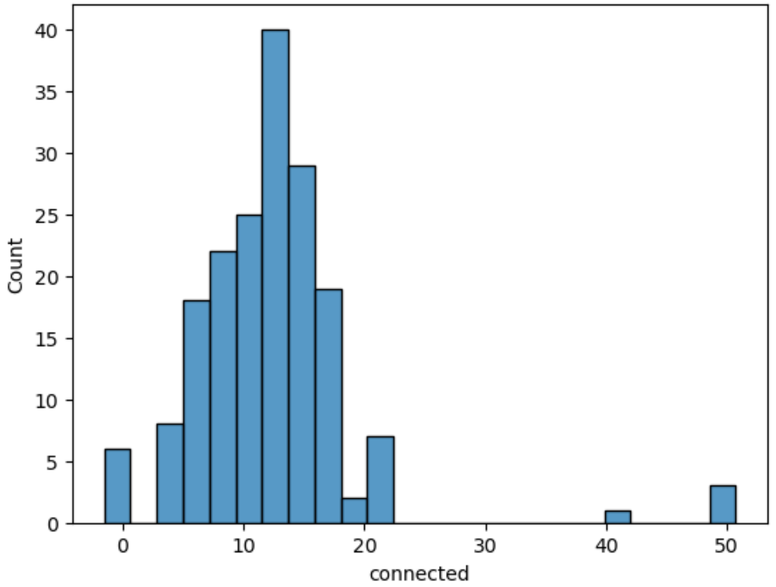
middle 50% of data

Statistics: IQR



Statistics: outliers





Statistics: Normalize

connected	number_logins
21.333161	2
3.120090	8
16.598087	3
10.711065	12
9.877369	11

Normalization (min-max scaler) - get data on a common scale

Standardization - rescale so that mean = 0, std = 1

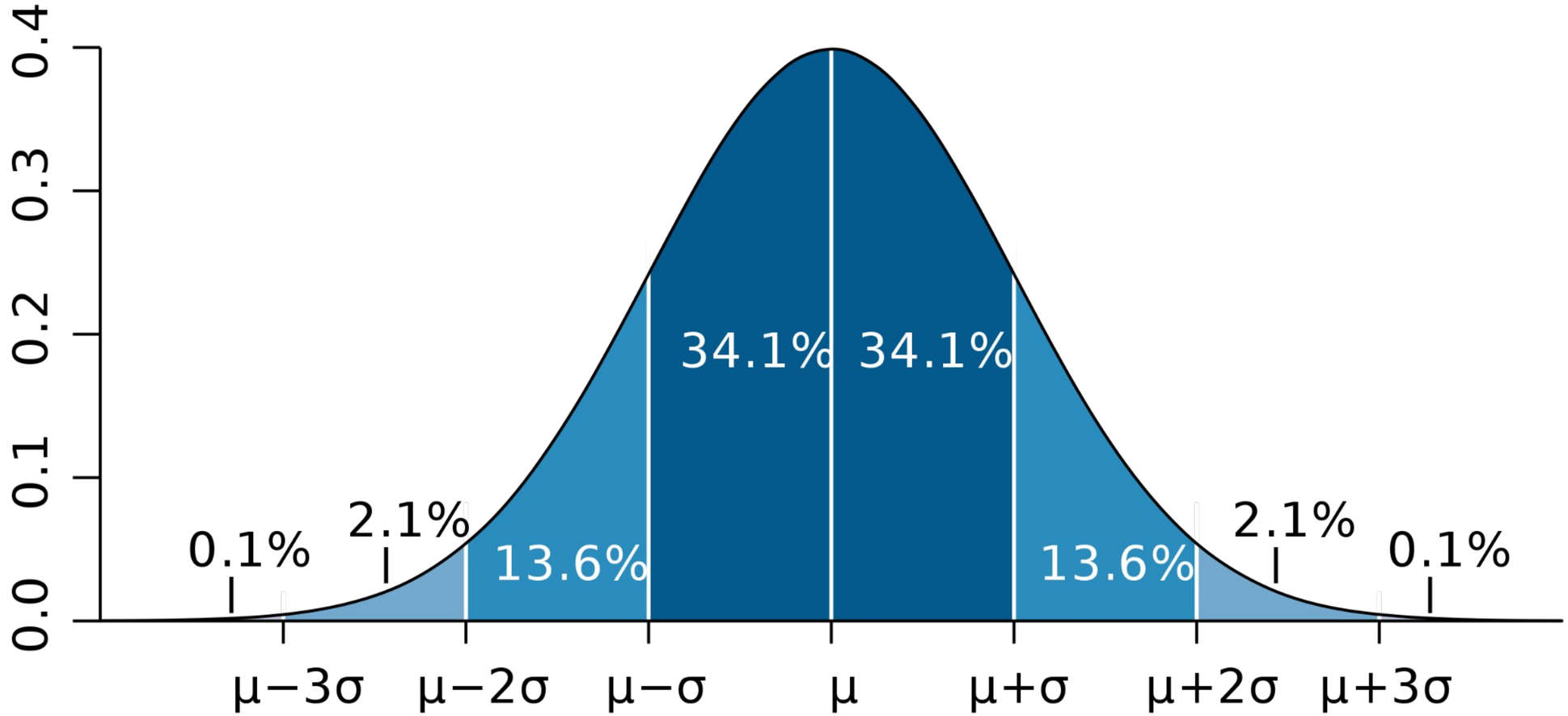
$$x_{scaled} = \frac{x - min}{max - min}$$

$$x_{scaled} = \frac{x - \mu}{\sigma}$$

	connected	number_logins
count	180.000000	180.0000
mean	11.980229	7.9500
std	7.269764	3.3095
min	-1.565569	1.0000
25%	7.834576	5.0000
50%	12.123636	8.0000
75%	14.088452	11.0000
max	50.770000	13.0000

Statistics: hypothesis testing

Normal distribution (Gaussian)



Ainali, CC BY-SA 3.0 <https://creativecommons.org/licenses/by-sa/3.0>, via Wikimedia Commons

`Series.abs()`	Absolute Value of the Series
`Series.count()`	Returns number of non-empty values in the series
`Series.max()`	Returns maximum value in the Series
`Series.mean()`	Returns the mean of a Series
`Series.median()`	Returns the median of a Series
`Series.min()`	Returns the minimum value in a Series
`Series.mode()`	Take a guess..
`Series.quantile([q])`	Returns the quantiles of a Series
`Series.sum`	Returns the sum of a series
`Series.std`	Returns the standard deviation of a Series

```
Series.describe(
)
```

Series.describe()

```
>>> random_numbers.describe()
count    50.000000
mean     50.620000
std      30.102471
min       1.000000
25%      25.500000
50%      54.000000
75%      73.000000
max      99.000000
dtype: float64
```

```
names = pd.Series(
   ['Jim','Bob',
   'Bob','Steve','Jim,'Jane','Steph','Emma','Rachel'])
names.describe()
```

```
names = pd.Series(
   ['Jim','Bob',
   'Bob','Steve','Jim,'Jane','Steph','Emma','Rachel'])
names.describe()
```

```
count        9
unique       7
top        Jim
freq         2
dtype: object
```

Finding Unique Values

```
unique_nums = []
for key, value in random_numbers.iteritems():
    if value not in unique_nums:
        unique_nums.append(value)
```

NO!!!

```
series.drop_duplicates()
series.unique()
```

Counting Unique Occurrences

```
series.drop_duplicates()
series.unique()
```

```
series.drop_duplicates()
series.value_counts(
series.unique()
)
```

```
series.drop_duplicates()
series.unique()
series.value_counts(bins=5
)
```

```
series.value_counts(bins=5
series.nunique()
series.unique()
series.duplicates()
normalize=True)
```

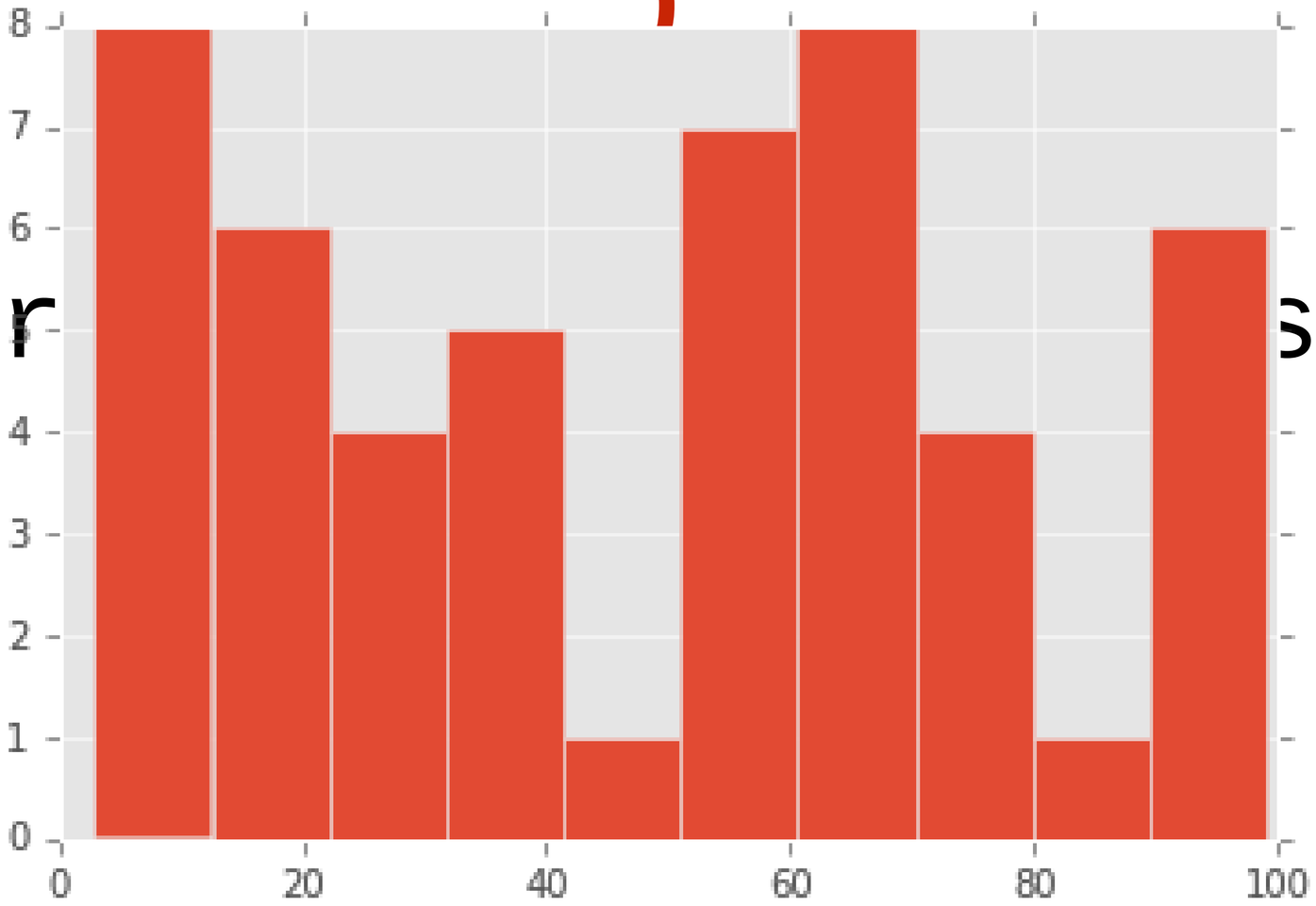
```
series.value_counts(bins=5
  series.drop_duplicates()
    series.unique()
      normalize=True,
       dropna=False)
```

```
In [7]: area_codes2.value_counts()

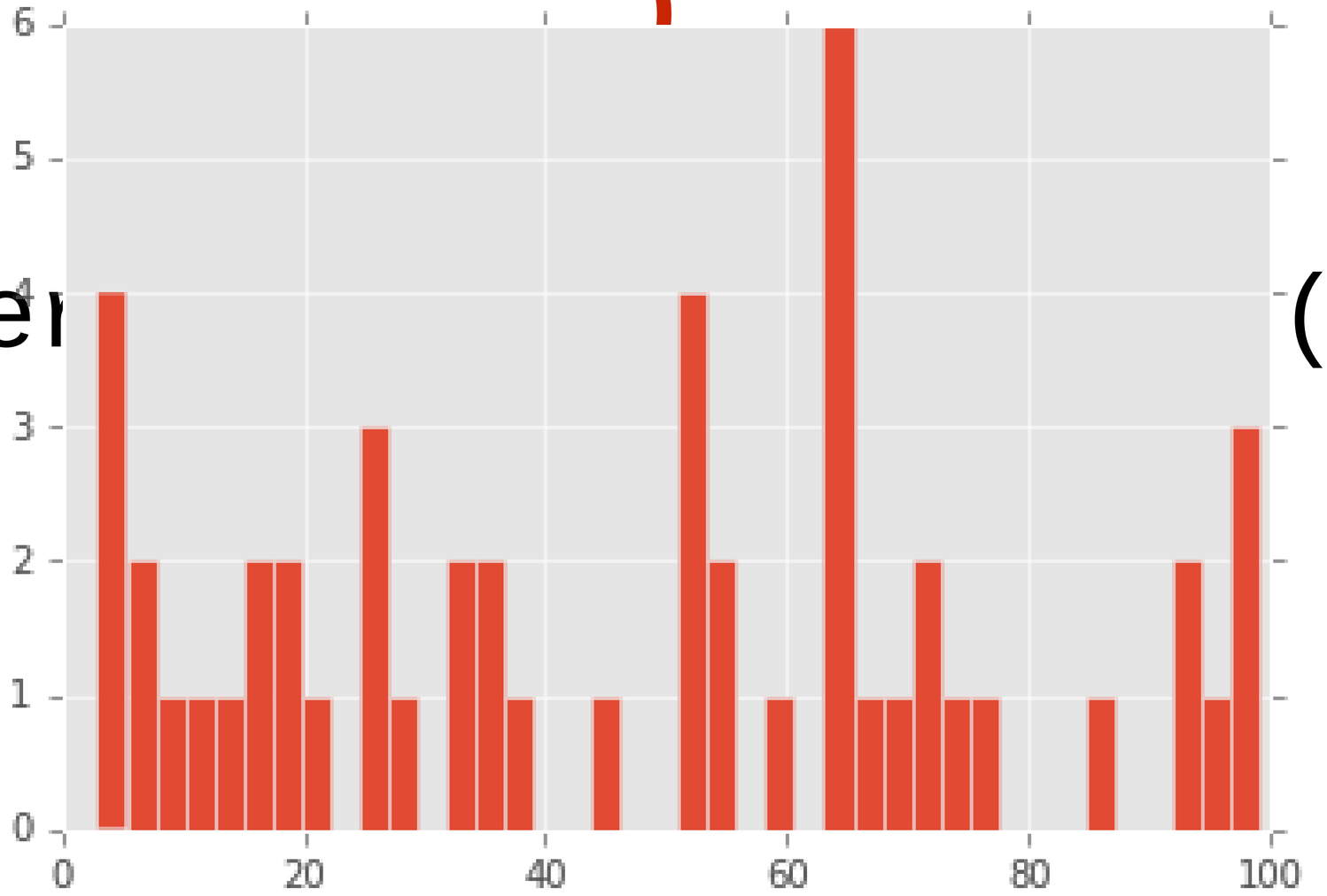
Out[7]: 833    7
        811    5
        822    5
        899    3
        844    3
        855    2
        dtype: int64
```

```
series.drop_duplicates()
series.hist(
series.unique()
)
```

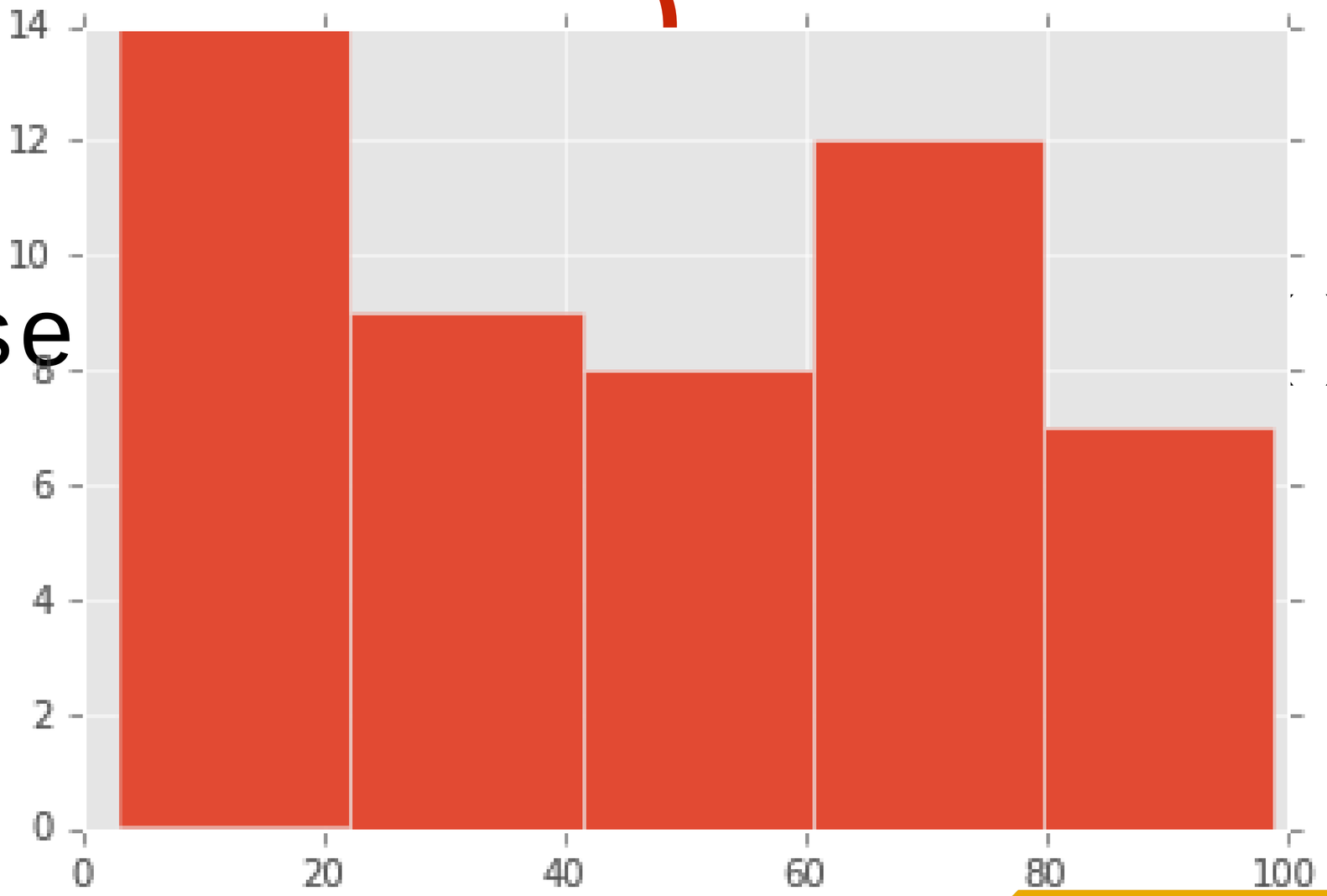
```
series.hist(
)
ser                                    s()
```



```
series.hist(bins=40
           )
ser                                    ()
```

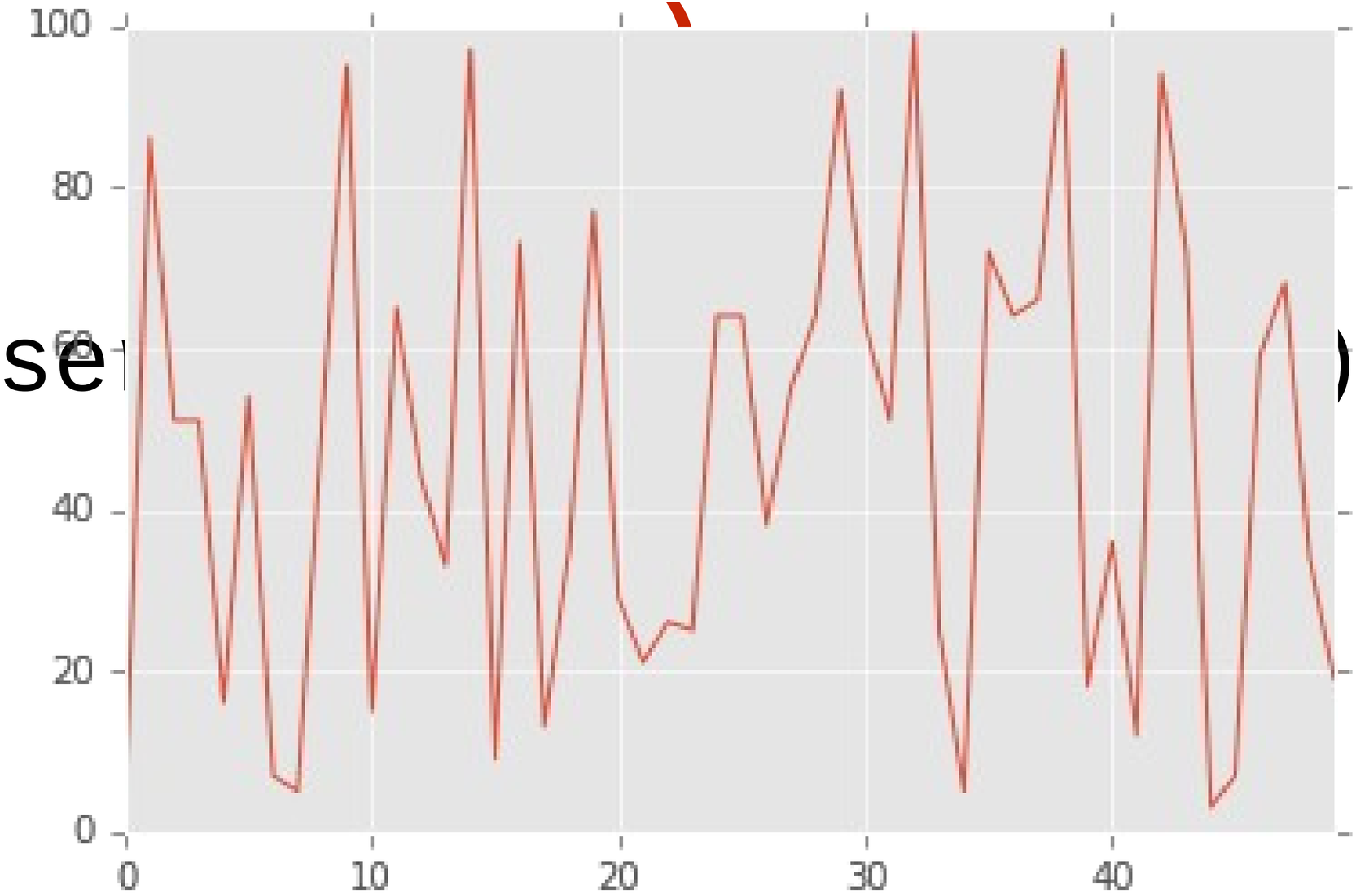


```
series.hist(bins=5
                  )
se                  )
```



```
series.drop_duplicates()
series.plot(
series.unique()
)
```

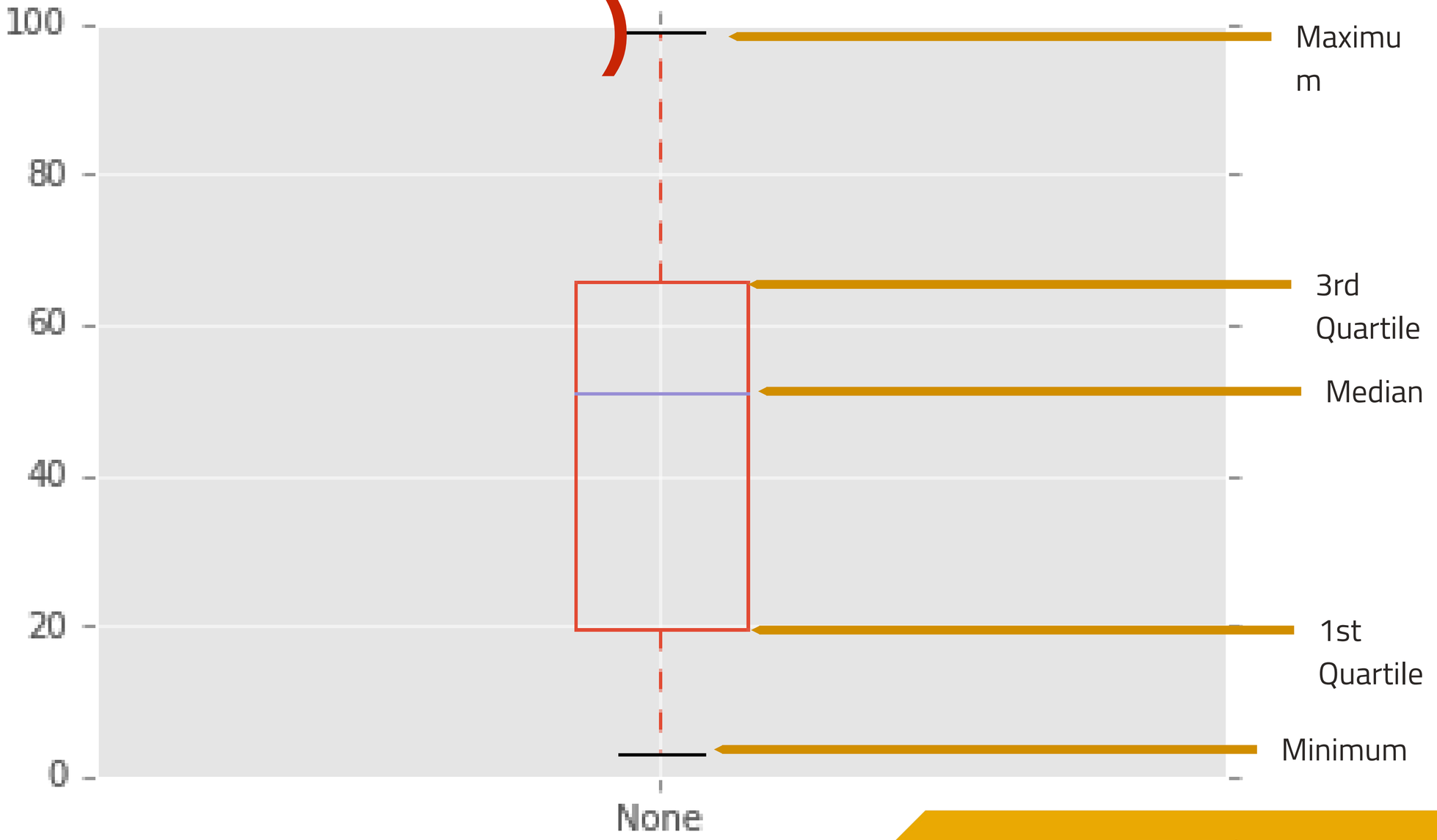
```
series.plot(
```




```
se
```

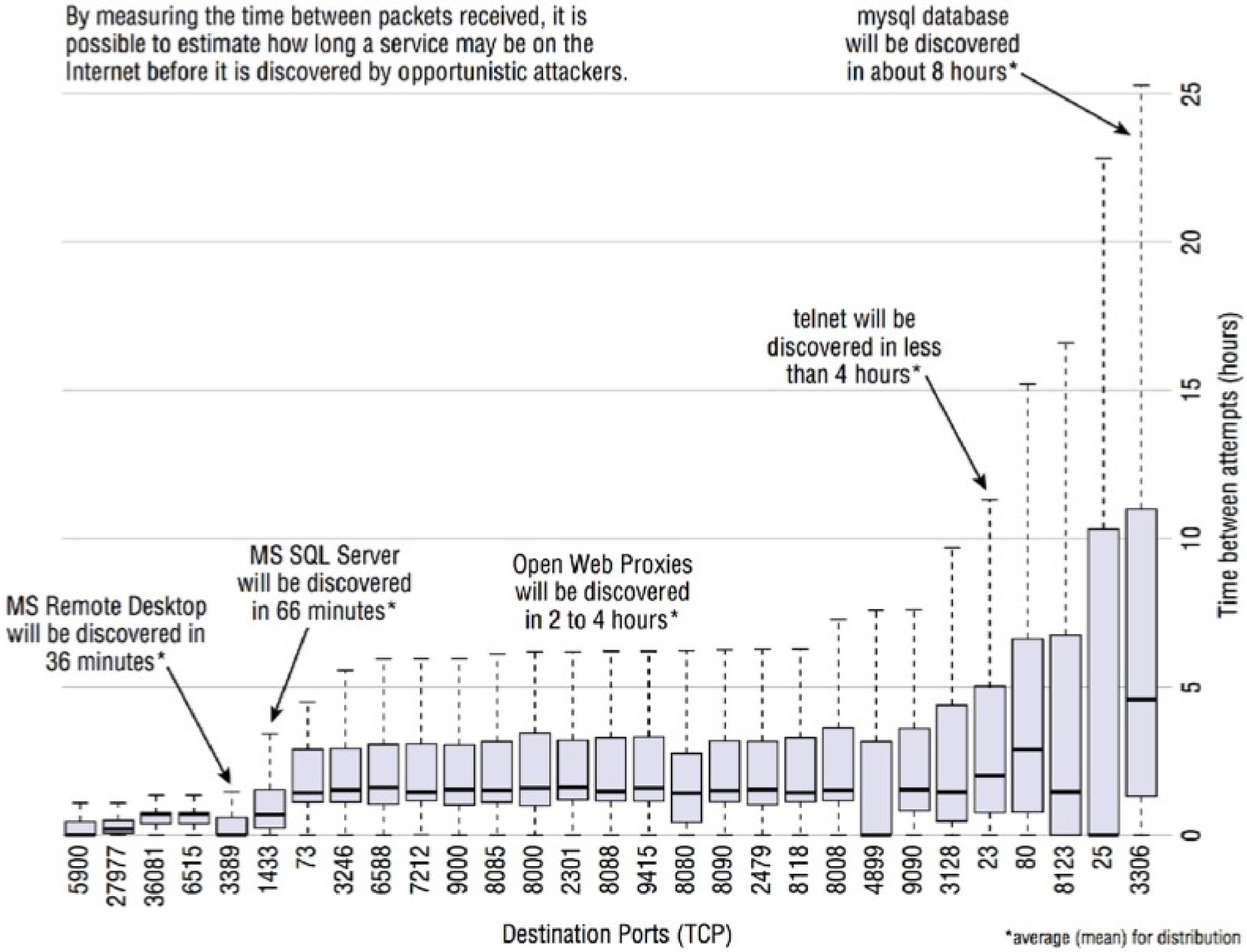
Reading a Boxplot

```
series.plot(kind='box')
```



How long will a service go undiscovered by opportunistic attackers?

By measuring the time between packets received, it is possible to estimate how long a service may be on the Internet before it is discovered by opportunistic attackers.



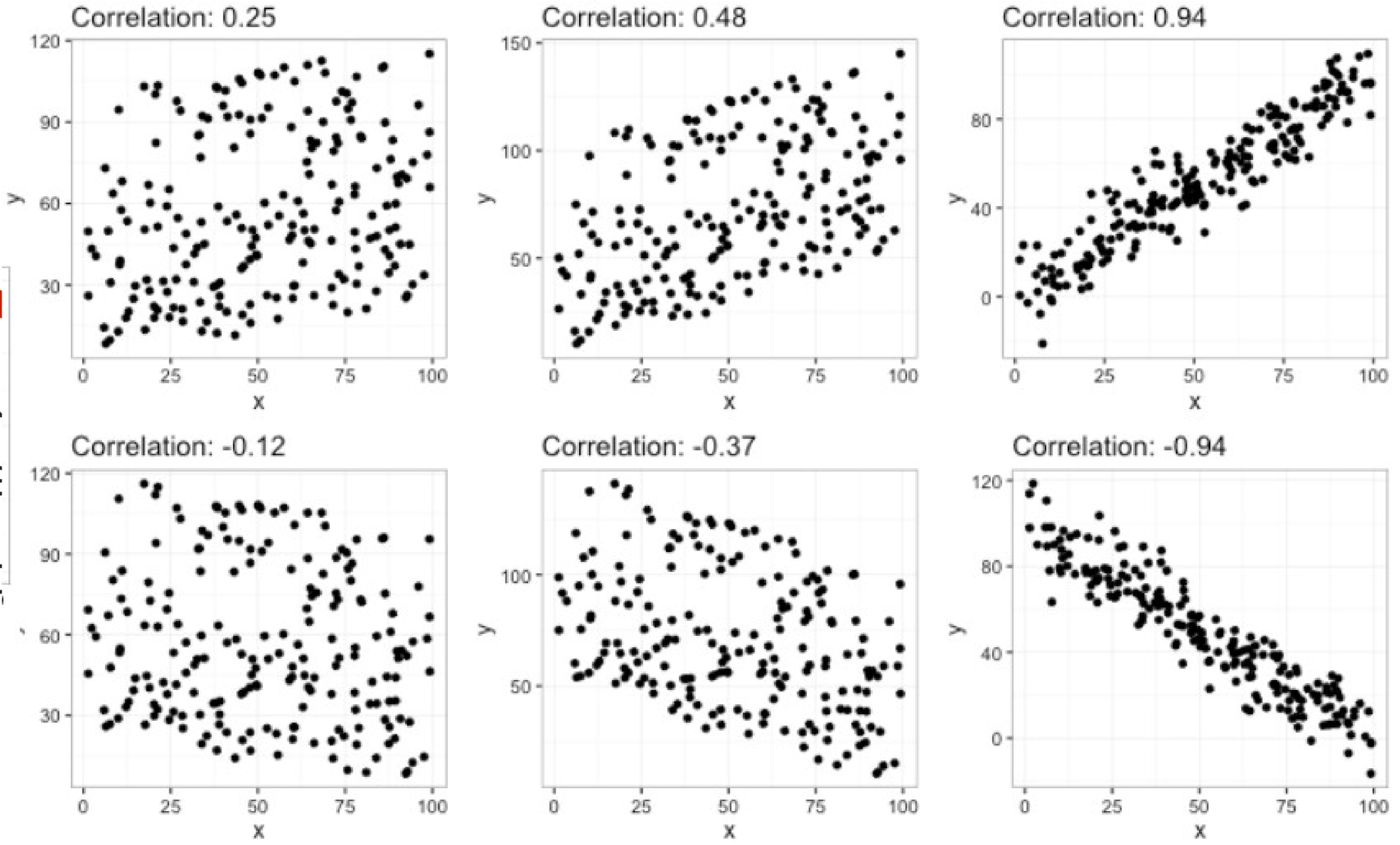
Correlations between Datasets

Correlations

- Measurement of the relationship between **two continuous variables**
- Output is between -1 and 1, with 1 being perfect correlation, -1 is perfect negative correlation, 0 is no correlation.
- Correlation measurement is referred to as r.

Correlations

- M
va
- Ou -1 is
pe
- Co



Correlations

- Measurement of the relationship between **two continuous variables**
- Output is between -1 and 1, with 1 being perfect correlation, -1 is perfect negative correlation, 0 is no correlation.
- Correlation measurement is referred to as r.

```
series1.corr( series2 )
```

Questions?

Part 2: Exploratory Data Analysis

Part 2: Two Dimensional Data

Data is Everything

- Data is the lifeblood of AI.
- Security data is often found in platforms such as ES, Splunk, PCAP and other platforms.
- The security world lives in Splunk (Ewwww!) but the data world lives in pandas data frames.

To build AI systems, you must be able to extract data from security platforms into data frames.

The Data Frame

Create a Pandas dataframe

```
df = pd.DataFrame( <data>, <index>, <column_names> )
```

DataFrames

Read in a CSV

```
df = pd.read_csv(<file>)
df = pd.read_csv(<url>)
```

DataFrames

Excel

```
df = pd.read_excel(<file>, sheetname=<sheetname>)
```

DataFrames

JSON

```
df = pd.read_json( <file> )
or
df = pd.read_json( <url> )
```

DataFrames

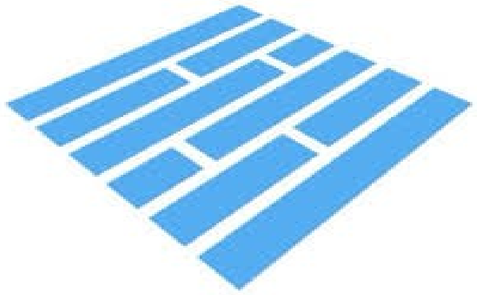
From a Database

```
df = pd.read_sql(<query>, <connection>)
```

DataFrames

Parquet

```
df = pd.read_parquet(<file>)
```



Log Files...

```
070823 21:00:32        1 Connect     root@localhost on test1
070823 21:00:48        1 Query       show tables
070823 21:00:56        1 Query       select * from category
070917 16:29:01       21 Query       select * from location
070917 16:29:12       21 Query       select * from location where id = 1 LIMIT 1
```

```
070823 21:00:32       1 Connect     root@localhost on test1
070823 21:00:48       1 Query       show tables
070823 21:00:56       1 Query       select * from category
070917 16:29:01      21 Query       select * from location
070917 16:29:12      21 Query       select * from location where id = 1 LIMIT 1
```

```
logdf = pd.read_table('../data/mysql.log',
names=['raw'])
```

```
070823 21:00:32       1 Connect     root@localhost on test1
070823 21:00:48       1 Query       show tables
070823 21:00:56       1 Query       select * from category
070917 16:29:01      21 Query       select * from location
070917 16:29:12      21 Query       select * from location where id = 1 LIMIT 1
```

```
logdf = pd.read_table('../data/mysql.log',
names=['raw'])
```

	raw
0	070823 21:00:32 1 Connect root@local...
1	070823 21:00:48 1 Query show tables
2	070823 21:00:56 1 Query select * f...
3	070917 16:29:01 21 Query select * f...
4	070917 16:29:12 21 Query select * f...

```
logdf = pd.read_table('../data/mysql.log', names=['raw'])
logdf['raw'].str.extract('(\d{6}\s\d{2}:\d{2}:\d{2})\s+(\d+)\s(\S+)\s(.+)', expand=False)
```

	0	1	2	3
0	070823 21:00:32	1	Connect	root@localhost on test1
1	070823 21:00:48	1	Query	show tables
2	070823 21:00:56	1	Query	select * from category
3	070917 16:29:01	21	Query	select * from location
4	070917 16:29:12	21	Query	select * from location where id = 1 LIMIT 1

```
logdf = pd.read_table('../data/mysql.log', names=['raw'])
logdf['raw'].str.extract('(?P<date>\d{6}\s\d{2}:\d{2}:\d{2})\s+
(?P<PID>\d+)\s(?P<Action>\S+)\s(?P<Query>.+)', expand=False)
```

	Date	PID	Action	Query
0	070823 21:00:32	1	Connect	root@localhost on test1
1	070823 21:00:48	1	Query	show tables
2	070823 21:00:56	1	Query	select * from category
3	070917 16:29:01	21	Query	select * from location
4	070917 16:29:12	21	Query	select * from location where id = 1 LIMIT 1

Web Server Logs



Web Server Logs

```
195.154.46.135 - - [25/Oct/2015:04:11:25 +0100] "GET
/linux/doing-pxe-without-dhcp-control HTTP/1.1" 200 24323
"http://howto.basjes.nl/" "Mozilla/5.0 (Windows NT 5.1;
rv:35.0) Gecko/20100101 Firefox/35.0"
```

Web Server Logs

```
195.154.46.135 - - [25/Oct/2015:04:11:25 +0100] "GET
/linux/doing-pxe-without-dhcp-control HTTP/1.1" 200 24323
"http://howto.basjes.nl/" "Mozilla/5.0 (Windows NT 5.1;
rv:35.0) Gecko/20100101 Firefox/35.0"
```

```
from apachelogs import LogParser
line_parser = LogParser("%h %l %u %t \"%r\" %>s
%b \"{Referer}i\" \"%{User-agent}i\"")
```

Web Server Logs

```
import apache_log_parser
line_parser = apache_log_parser.make_parser("%h %l %u %t
\"%r\" %>s %b \"%{Referer}i\" \”%{User-agent}i\”")
server_log = open(“../data/hackers-access.httpd”, "r")
parsed_server_data = []
for line in server_log:
    data = {}
    data = line_parser(line)
    parsed_server_data.append( data )
server_df = pd.DataFrame( parsed_server_data  )
```

Web Server Logs

er	request_first_line	request_header_referer	request_header_user_agent	request_http_ver	request_method	request_url
	GET /linux/doing-pxe-without-dhcp-control HTTP...	http://howto.basjes.nl/	Mozilla/5.0 (Windows NT 5.1; rv:35.0) Gecko/20...	1.1	GET	/linux/doing-pxe-v
	GET /join_form HTTP/1.0	http://howto.basjes.nl/	Mozilla/5.0 (Windows NT 5.1; rv:35.0) Gecko/20...	1.0	GET	/join_form
	POST /join_form HTTP/1.1	http://howto.basjes.nl/join_form	Mozilla/5.0 (Windows NT 5.1; rv:35.0) Gecko/20...	1.1	POST	/join_form
	GET /join_form HTTP/1.0	http://howto.basjes.nl/	Mozilla/5.0 (Windows NT 6.3; WOW64; rv:34.0) G...	1.0	GET	/join_form
	POST /join_form HTTP/1.1	http://howto.basjes.nl/join_form	Mozilla/5.0 (Windows NT 6.3; WOW64; rv:34.0) G...	1.1	POST	/join_form
	GET /acl_users/credentials_cookie_auth/require...	http://howto.basjes.nl/join_form	Mozilla/5.0 (Windows NT 6.3; WOW64; rv:34.0) G...	1.1	GET	/acl_users/creden

Nested Data?



Nested Data?


```
{"time": 1084443427.311224,
"timestamp": "2004-05-13T10:17:07.311224",
"IP": {

    "version": 4,
    "ttl": 128,
    "proto": 6,
    "options": [],
    "len": 48,
    "dst": "65.208.228.223",
    "frag": 0,
    "flags": 2, "src": “145.254.160.237",
    "chksum": 37355
    },
    "Ethernet": {"src": "00:00:01:00:00:00", "type": 2048, "dst":
    “fe:ff:20:00:01:00"},
    …
```

Nested Data

```
pd.read_json('nested_data.json'
)
```

```
pd.read_json('nested_data.json'
)
```

	DNS	Ethernet	IP	TCP	UDP	time	timestamp
0	NaN	{'type': 2048, 'dst': 'fe:ff:20:00:01:00', 'sr...	{'dst': '65.208.228.223', 'len': 48, 'version'...	{'window': 8760, 'chksum': 49932, 'sport': 337...	NaN	1.084443e+09	2004-05-13 10:17:07.311224
1	NaN	{'type': 2048, 'dst': '00:00:01:00:00:00', 'sr...	{'dst': '145.254.160.237', 'len': 48, 'version...	{'window': 5840, 'chksum': 23516, 'sport': 80,...	NaN	1.084443e+09	2004-05-13 10:17:08.222534
2	NaN	{'type': 2048, 'dst': 'fe:ff:20:00:01:00', 'sr...	{'dst': '65.208.228.223', 'len': 40, 'version'...	{'window': 9660, 'chksum': 31076, 'sport': 337...	NaN	1.084443e+09	2004-05-13 10:17:08.222534
3	NaN	{'type': 2048, 'dst': 'fe:ff:20:00:01:00', 'sr...	{'dst': '65.208.228.223', 'len': 519, 'version...	{'window': 9660, 'chksum': 43352, 'sport': 337...	NaN	1.084443e+09	2004-05-13 10:17:08.222534
4	NaN	{'type': 2048, 'dst': '00:00:01:00:00:00', 'sr...	{'dst': '145.254.160.237', 'len': 40, 'version...	{'window': 6432, 'chksum': 33825, 'sport': 80,...	NaN	1.084443e+09	2004-05-13 10:17:08.783340

Nested Data

```
from pandas import json_normalize
import json
import pandas as pd
with open('nested.json') as data_file:
    pcap_data = json.load(data_file)
df =
pd.DataFrame(json_normalize(pcap_data))
```

```
df =
pd.DataFrame(json_normalize(pcap_data))
```

...	TCP.seq	TCP.sport	TCP.urgptr	TCP.window	L
...	951057939.0	3372.0	0.0	8760.0	
...	290218379.0	80.0	0.0	5840.0	
...	951057940.0	3372.0	0.0	9660.0	
...	951057940.0	3372.0	0.0	9660.0	
...	290218380.0	80.0	0.0	6432.0	

ElasticSearch & Splunk



splunk>

ElasticSearch & Splunk

```
pip install
huntlib
```

```
e = ElasticDF(
 url="https://localhost:9200",
 ssl=True,
 username="myuser",
 password="mypass"
)
df = e.search_df(
 lucene="proto:tcp AND port:80",
 index="myindex-*",
 days=1,
 normalize=False
)
```

```
s = SplunkDF(
 host="<splunk_ip>",
 username="myuser",
 password="mypass"
)
df = s.search_df(
 spl='search index=win_events src="10.9.*.*"',
 start_time=datetime.now()-timedelta(days=2),
 end_time=datetime.now()
)
```

Manipulating a DataFrame

Creating a new Column

```
df['new_col'] =
df['column1'] + 3
```

Two Ways of Accessing Columns

```
my_series = df.loc[:,'column1']
```

Don't use the dots!

```
series = df.column1
```

Return a **series**

```
my_series = df.loc[:,'column1']
```

Return a **dataframe**

```
df = df[['column1','column2','column3']]
```

Filtering a DataFrame

```
df[<boolean condition>]
```

```
df[['col1','col2']][df['col3'] > 5]
```



Aggregations can be done over rows or columns

```
data.sum(axis='index')
```

OR

```
data.sum(axis='columns')
```

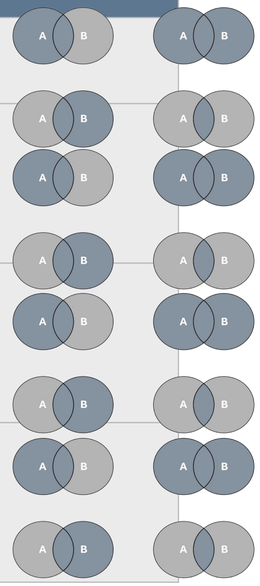
Common options for Pandas Methods

```
DataFrame.drop(labels,
                 axis='columns',
               level=None,
               in_place=False,
               errors='raise')
```

Merging Data Sets

```
combinedSeries = pd.concat(
                    [series1, series2])
```

```
pd.merge( leftData, rightData,
how="<join type>" )
```

Option	SQL Equivalent	Description	
inner	INNER JOIN	Intersection	
left	LEFT OUTER JOIN	Returns items in Set A, but not in Set B	
right	RIGHT OUTER JOIN	Returns items in Set B, but not in Set A	
outer	FULL OUTER JOIN	Returns the union of both sets	

```
pd.merge( leftData, rightData,
          how="<join type>",
          on=<field list> )
```

Grouping and Aggregating Data

Grouping and Aggregating Data

date	src_ip	dst_ip	port
2018-06-21	192.168.20.2	10.10.4.1	80
2018-06-21	192.168.20.1	10.10.4.2	443
2018-06-21	192.168.20.2	10.10.5.1	80
2018-06-22	192.168.20.2	10.10.4.1	80

```
df.groupby(['src_ip'
)
```

Grouping and Aggregating Data

date	src_ip	dst_ip	port
2018-06-21	192.168.20.2	10.10.4.1	80
2018-06-21	192.168.20.1	10.10.4.2	443
2018-06-21	192.168.20.2	10.10.5.1	80
2018-06-22	192.168.20.2	10.10.4.1	80

```
df.groupby('src_ip')
['port'].count()
```

Grouping and Aggregating Data

date	src_ip	dst_ip	port
2018-06-21	192.168.20.2	10.10.4.1	80
2018-06-21	192.168.20.1	10.10.4.2	443
2018-06-21	192.168.20.2	10.10.5.1	80
2018-06-22	192.168.20.2	10.10.4.1	80

```
df.groupby('src_ip')
['port'].count()
```

src_ip	count
192.168.20.1	1
192.168.20.2	3

Grouping and Aggregating Data

date	src_ip	dst_ip	port
2018-06-21	192.168.20.2	10.10.4.1	80
2018-06-21	192.168.20.1	10.10.4.2	443
2018-06-21	192.168.20.2	10.10.5.1	80
2018-06-22	192.168.20.2	10.10.4.1	80

```
df.groupby([‘date’,’src_ip’])
[‘port'].count()
```

Grouping and Aggregating Data

date	src_ip	dst_ip	port
2018-06-21	192.168.20.2	10.10.4.1	80
2018-06-21	192.168.20.1	10.10.4.2	443
2018-06-21	192.168.20.2	10.10.5.1	80
2018-06-22	192.168.20.2	10.10.4.1	80

```
df.groupby(['date','src_ip'])
['port'].count()
```

```
date        src_ip
2018-06-21  192.168.20.1
1
            192.168.20.2    2
2018-06-22  192.168.20.2
1
```

Grouping and Aggregating Data

date	src_ip	dst_ip	port
2018-06-21	192.168.20.2	10.10.4.1	80
2018-06-21	192.168.20.1	10.10.4.2	443
2018-06-21	192.168.20.2	10.10.5.1	80
2018-06-22	192.168.20.2	10.10.4.1	80

```
df.groupby(['date','src_ip'])
['port'].count()
```

Multi-Index

```
date          src_ip
2018-06-21    192.168.20.1
1
              192.168.20.2    2
2018-06-22    192.168.20.2
1
```

Grouping and Aggregating Data

date	src_ip	dst_ip	port
2018-06-21	192.168.20.2	10.10.4.1	80
2018-06-21	192.168.20.1	10.10.4.2	443
2018-06-21	192.168.20.2	10.10.5.1	80
2018-06-22	192.168.20.2	10.10.4.1	80

```
df.groupby(['date','src_ip'], as_index=False)['port'].count()
```

	date	src_ip	port
0	2018-06-21	192.168.20.1	1
1	2018-06-21	192.168.20.2	2
2	2018-06-22	192.168.20.2	1

```
my_series.dropna()
my_series.fillna(value=“<value to fill
with>")
```

Scaling Pandas

- Pandas is limited by the amount of memory on your machine. You will need 5-10x as much RAM as the size of your datasets.
- Operations on DataFrames creates copies of the data in memory.
- Pandas 3.0 is a major improvement in speed!

Scaling Pandas

Several drop-in or near drop-in replacements for Pandas exist that run on GPUs or parallelize operations.

- **Dask**: A partial Pandas replacement geared towards running code on compute clusters.
- **cuDF**: A drop in partial replacement for Pandas for use on GPUs. Developed by NVIDIA
- **Modin:** A drop in GPU based replacement for Pandas.
- **Ray:** A unified low level framework for scaling AI and Python applications.
- **RAPIDS:** A set of libraries and APIs that runs on NVIDIA CUDA GPUs. It offers GPU-enabled substitutes for popular Python libraries, such as cuDF for Pandas, cuML for scikit-learn, and cuGraph for NetworkX.
- **Polars:** A fast dataframe library implemented in Rust. (Runs in Python).
- **PySpark:** The Python API for Spark that lets you harness the simplicity of Python and the power of Apache Spark.

Apache Arrow

- Apache Arrow is a standardized framework for developing data analytics applications that use columnar data.
- Arrow can be used in python (PyArrow), Spark and many other languages.
- Arrow is designed as a compliment to parquet format.
- Main advantages are interoperability, speed and efficiency.

Questions?

In-Class Exercise

Please complete Worksheet 1.1: Exploratory Data Analysis